COMP4423 Assignment 2

PolyU Main Campus Image Classification

Name: Huang Haoran

Student number: 24101322D

Course: COMP4423 Computer Vision, 2026/27 Semester 1

Deadline: 23:59 HKT, 28 October 2026

Code: https://github.com/Retr0-rgb-lab/COMP4423_Assignment2

Dataset link (the same string as `DATASET_LINK.txt`):
https://drive.google.com/drive/folders/1_ld7gsHLhzyFJY3bUgTIwoOeVXRDFuAr

The complete photograph set is on Google Drive as 27 zip parts named `COMP4423_A2_dataset_part01.zip` through `COMP4423_A2_dataset_part27.zip`. They are one archive cut into 27 byte chunks, not 27 separate archives, so join them before unpacking: `cat COMP4423_A2_dataset_part*.zip > COMP4423_A2_dataset.zip` on Linux or macOS, `copy /b` on Windows, then `unzip COMP4423_A2_dataset.zip`. The file names inside are the ones used in this report. The link is shared as *anyone with the link*, so no sign-in is needed; each part is over 100 MB, so Google shows a *Download anyway* button that requires no account.

0. What the assignment asks, and where this report answers it

Requirement	Where it is answered
Task 1 --- class definitions, collection plan, and the planned locations on an annotated map or location list	Section 1.2 for the definitions, Section 2.1 for the plan, Figures 2 and 3 for the locations
Task 2 --- a labelled dataset of sufficient scale, split by location group, with the counts for every class and split	Sections 2.1 to 2.3 for the data, Section 2.5 for the split and the counts, Section 2.6 for the limits
Task 3 --- features, a traditional classifier, justified algorithmic choices, training evidence, reproducible code	Section 3, with the choice table in Section 3.1, the feature and classifier tables in Sections 3.3 and 3.4, and the training-evidence table in Section 3.6
Task 4 --- test metrics, prediction examples, and failure modes related to the limits of the chosen features	Section 4 for the metrics, Section 5 for the error analysis and the frame-by-frame reading
Task 5 --- a local application that loads the saved pipeline, uses the same preprocessing, answers from pixels, and is launched with clear instructions	Section 6, with the launches in 6.1, the window in 6.2, the prediction path in 6.3, the checks in 6.4 and two real runs in 6.5
Task 6 --- a report whose template questions are answered	Section 7 answers the six template questions one by one, Section 8 reflects on the result, and Section 11 is the glossary of terms

The six questions in the report template are the six subsection headings of Section 7, in the order the template gives them: how the program was designed and tested, how its robustness is ensured in a real scenario, what problems were found and how they were solved, how GenAI assisted the implementation, how GenAI understands and solves the tasks, and the limitations and areas for improvement in GenAI's solution. The questions are not paraphrased, so a reader can match them against the template one by one.

1. Introduction

1.1 Problem and motivation

The program takes one scene photograph and predicts whether the camera was inside the Hong Kong Polytechnic University main campus. The positive class is that main campus only. A building that belongs to PolyU but sits off the main campus is outside. The label is geographic. It is not ``indoors versus outdoors'', and it is not ``red brick versus something else''.

The practical question is whether a traditional classifier, reading pixels alone, can tell those two geographies apart when many scenes look alike: walkways, courtyards, lawns, and red-brick façades exist on both sides of the boundary. The Hong Kong University Main Building was collected for that reason. It is red brick, and it is outside PolyU.

1.2 Class definitions

The two classes are the ones the official campus map defines, with no local reinterpretation.

Label	Definition used in this project
inside (1)	The camera and the main scene are inside the PolyU main campus, using the official campus map as the boundary [10].
outside (0)	The camera and the main scene are outside that campus. Affiliated sites off the main campus stay outside.

Uncertain boundary views are excluded: gate thresholds, covered turnstiles at the entrance, connecting bridges, and shots that look across the boundary. The camera and the subject must be on the same side. Labels are not assigned from red brick, a logo, or architectural style. The classifier receives pixels only. GPS, EXIF, file names, folder names, and map lookups may be used to check a label or to find a file. They are not features.

At the main entrance, 11 photographs were removed under this rule: 4 taken at the campus name board, where the side of the gate is ambiguous, and 7 in the covered passage at the turnstiles, which is a threshold and also contains recognizable people.

1.3 Pipeline in one pass

1. Photographs are grouped by physical place. One place is one `group_id`.
2. Groups are split into train, validation, and test. A group is never split.
3. Each training and evaluation image is resized and center-cropped to 256×256 by `resize_and_crop` in `src/task3/preprocess.py`.
4. HOG is computed on the grayscale crop, with the block normalisation the descriptor applies internally. The random forest needs no further scaling, so the locked pipeline has no scaler; the linear SVM and logistic-regression rows do have one, fitted inside each fold. The locked model uses this descriptor alone; HOG+LBP, LBP alone, a colour histogram and two bag-of-words variants were also measured and are the comparison features of Section 4.1.
5. A random forest (300 trees, maximum depth 12) outputs outside or inside.
6. The fitted object is `models/task3_pipeline.joblib`.

Figure 1 shows the same path as a diagram, together with the order in which the model was chosen.

Two things about the figure are worth stating in words. The left-hand column is a single path: the photograph goes through one resizer, one feature extractor and one classifier, and the object that comes out is the file the application loads. There is no second path, and the application cannot reach the label any other way, because the label is produced by `predict_rgb` and by nothing else. The right-hand column is ordered: the feature comparison and the classifier comparison both run on the training places alone, in two stages, and only after both have finished is the test set read, once. The red line in the figure marks that boundary, and everything about how the result in Section 4 should be read follows from it.

Deep networks and CNN features are not used. The classifier reads pixels only.

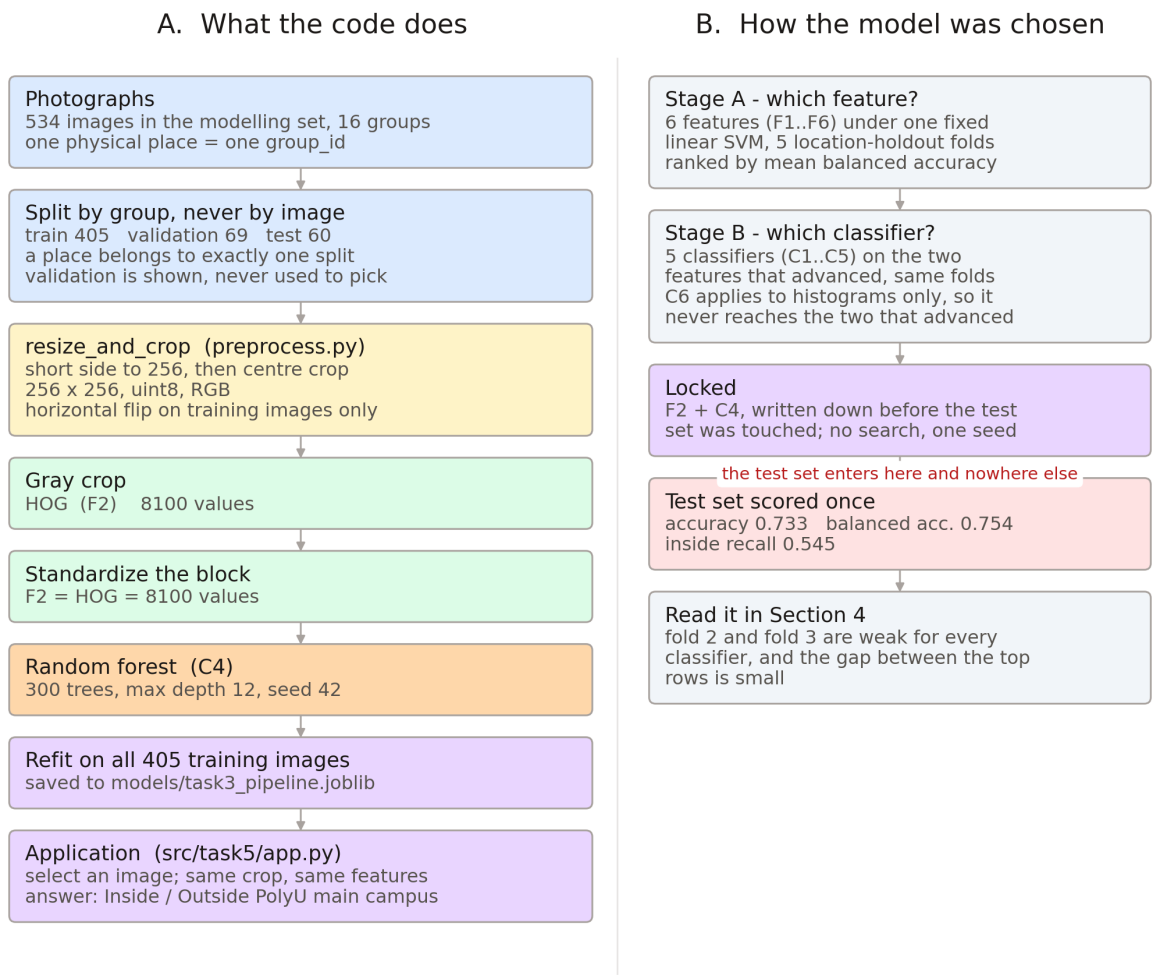


Figure 1. The pipeline, and the order in which the model was chosen. Left: what the code does, from the photographs through the split, the crop, the feature and the random forest to the saved pipeline that the application loads. Right: how the model was chosen. The comparison runs on the training places alone, in two stages, and the test set is reached once, after the choice has been written down. Six features were compared in Stage A and five classifiers in Stage B; the diagram names the winner, not every candidate.

1.4 Local application

Section 6 is the desktop program that loads `models/task3_pipeline.joblib` and shows one photograph's prediction. Sections 4 and 5 are the validation report and the single test evaluation of that same object. The application does not choose features and does not rescore the test set.

---

2. Dataset

2.1 Places and the collection strategy

One physical place is one location group. Nearby views of the same courtyard, building, or landmark stay together. Separate buildings in the same district do not. HKU's modern teaching blocks and the Main Building are different groups. West Kowloon's M+ museum, Art Park, and harbourfront promenade are different groups. Continuous PolyU walkways stay in one group.

Inside the group, variation is recorded in columns rather than new groups: `sub_area` for the part of the place, and `scene_class` for one of 13 scene types. The five shooting axes also stay inside the group. Making any one of them its own group would turn the split into a split on that axis.

Axis	How it was varied
Viewpoint	About six stations. Facing direction (front, 45 degrees, side), camera height (ground, slight upward, downward from steps), and horizontal placement of the subject (left, center, right).
Distance	Two stations, each with a near, middle, and far frame.
Close view	Wall--ground junctions and materials, with spatial depth kept in the frame. Isolated objects were not collected.
Lighting	Night frames kept in the daytime group of the same place, so day and night of one place cannot fall into different splits.
Background	At least three background kinds per group. The same kind is used on both sides of the label when possible.

The collection quotas were within-group targets, not the final counts. Final counts are below.

Both classes contain buildings, covered walkways, green space, and open paving. Eleven of the thirteen scene types appear on both sides of the label. The two types that appear only outside are deliberately placed there: the five indoor frames of the HKU corridor, so that ``indoors'' cannot become a synonym for inside, and the nine market-stall frames, six of them on Temple Street and three on the Jordan Road stretch, where a night market would otherwise be a free cue.

Among the 716 usable photographs, before the per-group cap:

Scene type	inside	outside
Building façade	67	82
Covered walkway	65	63
Urban distance view	22	54
Tree-lined path	27	48
Road frontage	10	57
Green space	20	35
Ramp or steps	33	15
Building entrance	31	9
Paved plaza	10	16
Construction hoarding	14	5
Water	5	14

Scene type	inside	outside
Market stall	0	9
Indoor space	0	5

Row sums equal 716. Façades and covered walkways are the two largest types (149 and 128), water has 19 photographs, road frontage is unbalanced (10 inside, 57 outside), and construction hoarding is temporary (14 inside, 5 outside). These limits are discussed in Section 2.6.

Night frames are 31 of the 304 usable inside images, 10.2%, and 56 of the 412 usable outside images, 13.6%. Every night frame sits in the daytime group of the same place, so day and night of one place cannot land in different splits. Within the modelling set they fall unevenly, and the placement matters for what the test score can be blamed on:

Split	inside, night / frames	outside, night / frames
train	21 / 194	30 / 211
validation	0 / 40	0 / 29
test	7 / 33	0 / 27
modelling set	28 / 267	30 / 267

All thirty outside night frames come from OUT-01 and OUT-03 and are in training. The seven inside night frames in test are all from IN-05, the main-entrance group, which is also the test inside place the model handles best, 13 of 16. The misses are concentrated in IN-07, which has no night frame at all, so the night share cannot account for them. Figure 2 puts the seven inside places on the campus map, and the table under it gives the usable count of each.

No.	Group	Place	Split	Usable images
01	IN-01	Walkway, J block to VA block	train	72
02	IN-02	Coffered walkway, P/Q block	train	62
03	IN-03	University Library, University Gallery and the E block	train	44
04	IN-04	FSCH, VS block and swimming pool	train	53
05	IN-05	Main entrance forecourt and clock tower	test	16

No.	Group	Place	Split	Usable images
06	IN-06	Building courtyard and Block A entrance	val	40
07	IN-07	CMA courtyard	test	17
		total, before the per-group cap		**304**

The counts are the usable images with status include; they sum to 304, the inside total before the 50-image cap and before class balancing. The map keeps the building letters of the official campus map, so a place can be found on the ground from the figures alone.

The campus map cannot show the outside class, which is spread over three districts of Hong Kong, so Figure 3 places all sixteen groups on one map. The nine outside groups are pinned on a city map; the seven inside groups are too close together for that scale, so they are pinned on a campus inset. A filled circle is a training group, a ring is a validation group and a square is a test group, and the colour is the class.

2.2 Hard negatives

Red is the easiest shortcut. Much of the main campus is red brick, and a color histogram can score well by voting ``red means inside''. Outside photographs were chosen to block specific inside appearances:

Outside group	Inside appearance it matches
OUT-06 HKU Main Building (test)	Red-brick façades at IN-01, IN-03, and IN-04. Edwardian red brick and granite. Paired with IN-05, the main entrance forecourt, which is also in the test set.
OUT-02 HKU modern blocks and links (train)	Coffered walkway IN-02 and covered links IN-06. Includes the five indoor corridors.
OUT-09 West Kowloon harbourfront (val)	Entrance steps IN-05 and the large stair at IN-06.
OUT-08 Art Park (train)	Open lawns and courts at IN-02, IN-06, and IN-07. Bound to the same split as OUT-05 because the two sites see each other across a road.
OUT-05 M+ (train)	Modern buildings and hard plazas.
OUT-04 Elements (train)	Large roofs over open space, compared with the coffered ceiling at IN-02.
OUT-03 Kowloon Park (train)	Lawns, courtyards, and water. Eleven red-tile Chinese-garden frames are an extra red-brick negative.
OUT-07 HKU campus green (test)	Lawns at IN-05 and the courtyard green at IN-07. Not a red-brick negative. Paired with IN-07 in the test set.

Outside group	Inside appearance it matches
OUT-01 Jordan (train)	Dense street façades, as a contrast to campus blocks.

2.3 Labels, exclusions, and anchors

734 photographs were annotated. 18 were excluded. 716 remained, in 16 places (7 inside, 9 outside). After a cap of 50 images per group and class balancing, the modelling set has 534 images, 267 inside and 267 outside. Section 2.7 records why the count changed on 2026-10-27.

Excluded	Count	Reason
IN-05	11	Name board at the gate (4) or covered turnstile passage (7). Notes on the manifest rows match this split.
IN-01	2	The subject is the Communal Building, which no location group covers, and the Library block is in the background; the place could not be established from the frame (Section 2.7).
OUT-01	2	A readable license plate in the foreground. Both frames are excluded on privacy grounds whatever the plate says; the readings are kept on the manifest rows.
OUT-02	2	Street signs on the outer side of the campus boundary.
OUT-03	1	Several recognizable faces.

Exclusion was decided on the photograph, and never on the strength of a model score.

Phone transfer stripped EXIF GPS. Ground truth anchors are signs that are visible in the frame, including the university library sign and crest (IN-03 `IMG_1459`), Stanley Ho Building (IN-04 `IMG_1229`), CMA Building and Lui Che Woo Building (IN-07 `IMG_1428`), ``THE UNIVERSITY OF HONG KONG'' on the Main Building (OUT-06 `IMG_0863`), and Jordan Road (OUT-01 `IMG_1002`). Every anchor is a sign that is visible in the frame, so each label can be re-checked from the photograph alone. The campus name board `IMG_1304` is excluded and is not used as an anchor.



Figure 2. The seven inside places on a PolyU campus map, one numbered pin per place. The pin numbers are the row numbers in the table below, which also gives the group id, the split and the usable count. Marker 05 is on the south side of J Core; the place it marks is the main-entrance forecourt with its inner lawn and clock tower. The 11 ambiguous gate frames are in the excluded set; the 16 frames that remain are on the campus side of the boundary, 14 of them the lawn garden and the clock tower.

2.4 Why this scale is enough, and where it is not

There is no required image count. The unit that has to be learned and tested is the place, not the frame. Sixteen places are enough to put several places of each class in training, hold out paired places for a five-fold location check, and still leave a validation pair and a test pair. Adding more frames inside a place that is already in training does not add an unseen place.

The cap and the balance step exist so that one large place cannot dominate the pixel count. OUT-01 entered with 82 usable frames and is stored as 50. The smallest place, OUT-07, has only 7 frames, which also makes the image counts uneven: the test outside side holds 27 images against 33 inside images. Sixteen places are enough to run the protocol. They are not enough to make the test estimate sharp: the test set can hold only four places.

2.5 Split, leakage controls, and counts

`data/annotations/splits.csv` is the split. It is not redrawn by hand: `src/tools/build_dataset.py` reads the annotation manifests, caps each group, balances the two classes, and

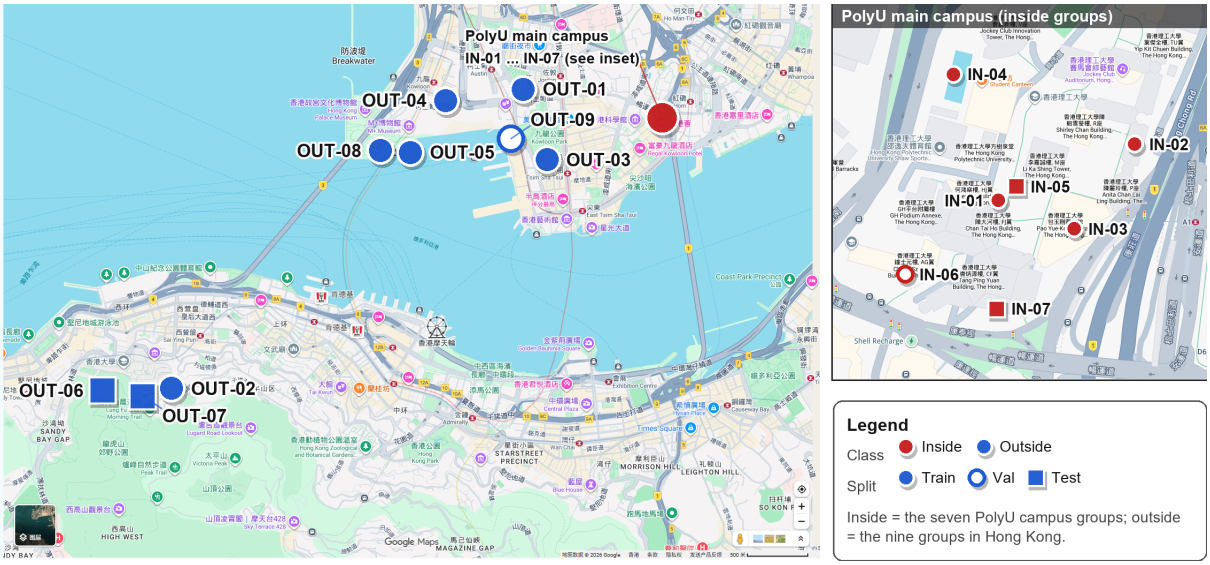


Figure 3. The sixteen collection locations. Left, Hong Kong, with the nine outside groups pinned from the University of Hong Kong in the south-west through Jordan Road and Kowloon Park to West Kowloon. Right, a campus inset with the seven inside groups. Colour is the class and shape is the split, so the reader can see that no outside place sits inside the campus boundary. The base map is a screenshot of Google Maps, and every pin is placed from a named feature of the map for that place rather than from a guessed coordinate.

assigns the groups to splits, and it refuses to finish if a group crosses splits, if OUT-05 and OUT-08 are separated, or if any split is missing a class. Augmentation is horizontal flip, applied only to training images after the split. Subsampling inside a capped group is evenly spaced so the viewpoint range remains.

The split below was re-derived on 2026-10-27, after a photo-level re-check found three grouping errors across four groups. Section 2.7 records what was found and what was done about it. The rule that assigns the groups is the project's own: the training split takes the largest groups, the remaining groups are handed out between validation and test so that the two stay as close as possible, and the two mutually visible West Kowloon groups are bound into one unit. The repair changed the split of three groups and the stored size of five, and the size is what that rule reads, so the assignment moved with it.

Image counts and groups, from `data/annotations/dataset_summary.md`:

Split	Class	Images	Share of the 534	Groups	Share of the 16	Group ids
train	outside	211	39.5%	6	37.5%	OUT-01, OUT-02, OUT-03, OUT-04, OUT-05, OUT-08
train	inside	194	36.3%	4	25.0%	IN-01, IN-02, IN-03, IN-04

Split	Class	Images	Share of the 534	Groups	Share of the 16	Group ids
val	outside	29	5.4%	1	6.3%	OUT-09
val	inside	40	7.5%	1	6.3%	IN-06
test	outside	27	5.1%	2	12.5%	OUT-06, OUT-07
test	inside	33	6.2%	2	12.5%	IN-05, IN-07
all	outside	**267**	**50.0%**	**9**	**56.3%**	OUT-01 … OUT-09
all	inside	**267**	**50.0%**	**7**	**43.8%**	IN-01 … IN-07

Totals: train 405, validation 69, test 60, sum 534. By images that is 405/534, 69/534, and 60/534 (about 75.8%, 12.9%, and 11.2%). By groups it is 10/16, 2/16, and 4/16, which is 62.5%, 12.5%, and 25%. The target is a split of location groups aiming at 60/20/20, with both classes in every split. The group shares can only be approximate: 20% of 16 groups is 3.2, and a group cannot be divided, so the split lands at 10, 2 and 4. Train and test come out above their targets, at 62.5% and 25%; validation falls below at 12.5%, for the reason given next. Both classes are present in every split, and the group shares are the ones to read. The test outside side is two groups, OUT-06 and OUT-07, because each test outside place is paired with a test inside place that resembles it (Section 2.2). OUT-07 contributes 7 of the 27 test outside images, and the validation outside side is the single group OUT-09.

The validation split is the one that falls below its target, at 2 of 16 groups, and the reason is worth stating plainly. There are 7 inside groups and 9 outside groups, and a validation split that holds out one group per class costs one group from each side. Holding out two groups per class instead would leave 3 inside and 5 outside groups for training, $7 - 2$ test $- 2$ validation and $9 - 2$ test $- 2$ validation. The alternative was rejected because it would have taken a second group out of training on each side, and the cost of the current split is recorded here rather than argued away: the validation score rests on one place per class and is therefore reported, and not used, as the selection score.

Stored counts per place (original usable $\rightarrow$ kept):

Group	Place	Class	Usable	Kept	Split
IN-01	Walkway, J Core to VA	inside	72	50	train
IN-02	P/Q coffered walkway	inside	62	50	train
IN-03	University Library, Gallery and E block	inside	44	44	train

Group	Place	Class	Usable	Kept	Split
IN-04	FSCH / VS / pool	inside	53	50	train
IN-05	Main entrance forecourt and clock tower	inside	16	16	test
IN-06	Block court and Core A entrance	inside	40	40	val
IN-07	CMA Building courtyard	inside	17	17	test
OUT-01	Jordan	outside	82	50	train
OUT-02	HKU modern blocks and links	outside	57	50	train
OUT-03	Kowloon Park	outside	72	32	train
OUT-04	Elements	outside	62	29	train
OUT-05	West Kowloon, M+	outside	21	21	train
OUT-06	HKU Main Building	outside	20	20	test
OUT-07	HKU campus green	outside	7	7	test
OUT-08	West Kowloon, Art Park	outside	41	29	train
OUT-09	West Kowloon har-bourfront	outside	50	29	val

Place names in this table are the full names. The generated summary in `data/annotations/dataset_summary.md` uses shorter labels for a few outside rows; the group id, class, counts, and split are the summary's.

2.6 Dataset limits that affect later numbers

- The test estimate rests on four places. A single difficult building moves the accuracy.
- OUT-07 contributes only 7 test images.

- Road frontage is 10 inside versus 57 outside in the usable pool. A model could treat heavy traffic as outside. The four test places are a forecourt, a courtyard, a brick building, and a lawn, so Section 5 does not attribute the test misses to that imbalance.
- Indoor evidence exists only on the outside class, and only as 5 frames.
- Construction hoarding will disappear. It is a durability limit, not a stable campus cue.
- Night shares are 10.2% inside and 13.6% outside of the usable pool, within a few points of each other.

2.7 A location group that was not one place

The group labels were assigned from the collection folders, and each group name followed its folder. A photo-level re-check on 2026-10-27, in which every frame of those four groups was looked at again, found three errors: two groups each held frames from more than one place, and a pair of groups photographed the same building from two different splits:

- **IN-05** mixed three places. Nine of its frames are the J-to-VA walkway past the Shaw Amenities Building and the Innovation Tower plaza, which is IN-01's place; thirteen are the University Library, the University Gallery, the E block and the plaza lawn in front of them, which is IN-03's place; and the rest are the main entrance, the name wall and the clock tower, which are its own.
- **IN-01** held six frames of the Library block, the Shirley Chan Building and the Chow Yei Ching Building.
- **OUT-08** and **OUT-09** both photographed the FREESPACE building at Art Park, one of them from the training split and one from the validation split.

That is leakage in the sense the assignment means it: the same scene on both sides of a split, and in the first case between the validation split and the test split, so any test score computed on that grouping carried a risk of optimism, and the size of that risk was never measured on its own. It also broke the project's own rule, which binds two places that see each other into one split unit.

The repair was made at the level of the annotation manifests, not by hand on the split table. `src/tools/recut_locations.py` holds the table of 31 frames that move and the two frames that are dropped, and rewrites the `group_id`, place name, sub-area and scene class of those rows. The split table and the summary are then re-derived by `build_dataset.py` from the repaired manifests, and the training pipeline was re-run on the result. A frame whose place could not be established from the photograph was removed rather than left in a group, because a frame that might belong to another place is exactly what puts the same scene on two sides of a split.

Two consequences are worth stating because they change how the numbers in Section 4 should be read. The first is that the group counts and the test set are those of the repaired split. Thirty-one frames changed group and two were excluded, and the kept pool is not 540 minus two: the moves change which frames the 50-per-group cap keeps, the two excluded frames came out of a group that is capped anyway, and the class balance is re-derived. The two sides settle at 267 inside and 267 outside. The second is that the Library complex is now a training group, so the test inside places are the main entrance and the CMA courtyard. The split was re-derived by the project's own rule rather than adjusted by hand: `build_dataset.py` gives the training split the largest groups and hands the rest alternately to validation and test. The repaired group sizes are what that rule reads, so the assignment moved with them: IN-03 went to training and IN-06

to validation. That is the rule's decision, not a choice made after seeing a score, and Section 11.6 lists the sizes it read.

---

3. Method

3.1 Design

The comparison was fixed before the run: the six features, the classifiers, the five folds, the single selection score, and the tie-break rules are in the shipped code, in `src/task3/config.py` and `src/task3/cv.py`. `cv.py` also writes a fingerprint of that source and of `splits.csv` beside the result tables, and refuses to reuse a table whose fingerprint does not match, so a result cannot silently come from code other than the code that ships.

Stage A asks which representation helps. Six hand-crafted features share one classifier: standardization, where the classifier needs it, and a linear SVM with `C = 1`. Stage B asks whether the classifier matters. The two features with the highest location-holdout score are each trained with the classifiers in Section 3.4. Parameters are fixed. There is no search on the validation set and none on the test set.

The selection score of a configuration is the mean balanced accuracy of five location-holdout folds. Balanced accuracy is the average of the outside recall and the inside recall, so a fold that only recognizes one class does not look successful. Validation accuracy is written down and is not part of the sort. The sort key in `src/task3/cv.py` is that mean balanced accuracy; it was read back from the code and checked against the published tables.

Ties on features keep the smaller dimension, then the earlier id. Ties on classifiers keep the simpler one, in the order logistic regression, linear SVM, naïve Bayes, k-NN, random forest, gradient boosting. One planned row was conditional on the winner being a linear SVM --- a 100-component PCA followed by an RBF SVM, fit inside the fold --- and it was not run, because the winner was not.

The table sets out the choices that decided the pipeline, the alternatives that were considered, and the evidence each answer rests on. Where the project cannot support an answer, the cell says so instead of asserting one.

Decision	Alternatives considered	Why this one	Evidence
The split unit is the location group	random per-image split; split by scene class	The model must work on a place it has not seen, so a place is what gets held out.	`build_dataset.py` refuses a group that crosses a split; Section 2.5
Traditional ML only, no neural network	pretrained CNN features; fine-tuning	The assignment excludes deep learning for Task 3.	Assignment requirements

Decision	Alternatives considered	Why this one	Evidence
256×256 centre crop	stretch to 256×256; multi-scale; 224 or 320 px	HOG's cells and blocks fix the crop geometry, and one `resize_and_crop` serves training, evaluation and the app, so the three cannot disagree. Cost: a portrait frame loses strips top and bottom.	`config.py`, `preprocess.py`; Section 6.4
HOG on the grayscale crop	HOG+LBP; LBP; colour; bag of words; BoW + pyramid	Shape beat the rest on the holdout: F2 0.701 against F3 0.607, F6 0.544, F1 0.537, F5 0.531. Adding LBP moved it +0.007 (SVM) and −0.009 (forest), both inside the fold spread.	Section 4.1; `features.py`
Rank by location-holdout balanced accuracy	validation accuracy; test accuracy; plain accuracy	The score must be measured on unseen places, and balanced accuracy stops a one-class answer from looking good.	`cv.py`; Section 4.1
Two stages: features, then classifiers	the full 6×6 grid; random search	Stage A moves the representation with the classifier fixed, Stage B the reverse. The representation moves the score more than the classifier does.	Sections 4.1, 4.2
No hyperparameter search	grid over `C`, depth, trees, HOG cell, LBP radius, vocabulary	Fixed before the run to keep the protocol pre-registered and single-seed. The F2 fold spread, 0.471 to 0.860, dwarfs the 0.009 between the top rows, so no row is claimed optimal and tuning was not measured.	Section 4.2

Decision	Alternatives considered	Why this one	Evidence
Horizontal flip, training only, after the split	colour jitter; rotation; scale; none	An augmentation that moves the camera would blur the place boundary under test; colour jitter would hide the lighting axis that was collected.	`cv.py`; Section 2.1

3.2 Preprocessing

`resize_and_crop` is the only resizer.

1. Open the file as RGB with Pillow. OpenCV's BGR reader is not used, so a later interface cannot silently disagree with evaluation.
2. Scale the short side to 256 with area interpolation, keeping the aspect ratio.
3. Center-crop to 256×256. A landscape frame loses equal strips on the left and right. A portrait frame loses equal strips on the top and bottom.
4. Return `uint8` of shape 256×256×3. HOG, LBP, and SIFT convert that crop to gray. The color histogram uses HSV of the same crop.

Horizontal flip is a training augmentation only. Validation and test images are not flipped. Color jitter is not used. Night and lighting were collected on purpose; jitter would hide that axis.

The random seed is 42.

3.3 Features

Id	Feature	Setting	Dimension
F1	HSV histogram	H 16 × S 16 × V 8. Whole frame plus a 2×2 grid. Each histogram L1-normalized, then concatenated.	10240
F2	HOG	9 orientations, 16×16 cells, 2×2 blocks, L2-Hys, on gray [1].	8100
F3	LBP	Uniform pattern, P = 8, R = 1, 10 bins [2]. Whole frame plus 2×2, each block L1-normalized.	50

Id	Feature	Setting	Dimension
F4	HOG + LBP	F2 and F3 standardized separately on the training images, then concatenated.	8150
F5	Bag of words	SIFT [4] keypoints, at most 500, in a bag-of-words histogram [5]. Mini-batch k-means, 256 words, fit on that fold's training descriptors only. Word counts L1-normalized. An image with no keypoints becomes a zero histogram and is counted in the log.	256
F6	Bag of words + spatial pyramid	F5 on the whole frame, plus four histograms on a 2×2 grid [6].	1280

SIFT is `cv2.SIFT_create` from `opencv-contrib` [9]. If that function is missing, the run stops. ORB is not a substitute. In the environment used for the 2026-10-27 run, SIFT was available.

The vocabulary is not stored across folds. A fold that is building a codebook never sees the held-out place.

What each of the six measures is set out in Section 11.1.

3.4 Classifiers

Id	Classifier	Setting
C1	Linear SVM	`LinearSVC`, `C = 1`, standardized features
C2	Logistic regression	`C = 1`, `lbfgs`, standardized features
C3	k-NN	k = 5. Cosine distance on histogram features. Euclidean distance when the feature includes HOG.
C4	Random forest	300 trees, `max_depth = 12`, `random_state = 42` [3]

Id	Classifier	Setting
C5	Histogram gradient boosting	`max_depth = 6`, `random_state = 42`
C6	Multinomial naïve Bayes	`alpha = 1`, only for non-negative L1 histograms (F1, F3, F5, F6)

C1, C4, and C5 are the three main families (SVM, random forest, gradient boosting). C2 and C3 are different decision rules. C6 is a simple histogram control. F2 and F4 skip C6 because they are not non-negative histograms.

C6 was additionally evaluated on the four non-negative L1 histogram features (F1, F3, F5, F6) as a supplementary comparison. Its results are the C6 columns of the table below. They do not enter the feature ranking, which is computed from the C1 column alone, so the locked pipeline is unaffected.

Feature	C1 linear SVM	C6 naive Bayes	C6 inside recall	C6 outside recall
F1 colour histogram	0.537	0.532	0.224	0.839
F3 LBP	0.607	0.500	0.000	1.000
F5 bag of words	0.531	0.518	0.048	0.988
F6 bag of words + spatial pyramid	0.544	0.576	0.419	0.733

What each of the six classifiers does is set out in Section 11.2.

The result is more mixed than a single collapse, and it depends on which feature the classifier is given. On F3 and F5 naive Bayes does collapse: inside recall is 0.000 and 0.048, so it answers outside for almost every campus frame, and its balanced accuracy settles at 0.500 and 0.518, near the score of a constant. On F1 and F6 it does not: inside recall is 0.224 and 0.419, and its balanced accuracy of 0.532 and 0.576 is close to or above what the linear SVM gets on the same feature, 0.537 and 0.544. Its best row is F6 at 0.576, which is fourth of the ten feature--classifier pairs measured in Stage A.

The outcome for the locked pipeline does not depend on which protocol is applied. The two features that advance are F4 and F2 under both, because every naive Bayes row stays below 0.607 and the linear SVM reaches 0.708 and 0.701 on those two. The order of the remaining features does change: the linear SVM puts F3 (0.607) first of the rest, while naive Bayes puts F3 last of all ten at 0.500 and raises F6 above the linear SVM's 0.544. So the supplementary pass agrees on the decision and disagrees on almost everything below it, which is the reason it is reported and not used for selection.

3.5 Location holdout

Each training place is a single class, so a fold always holds out one inside place and one outside place together. OUT-05 and OUT-08 are one atom. The five folds are written in `src/task3/config.py`:

Fold	Held-out inside	Held-out outside
0	IN-01	OUT-01
1	IN-02	OUT-02
2	IN-03	OUT-03
3	IN-04	OUT-04
4	IN-01	OUT-05 and OUT-08

IN-01 is held out twice because there are five outside atoms and four inside training groups. The five balanced accuracies are averaged as they are. The two IN-01 folds are not merged. The folds follow the training places in group order, so they are re-derived whenever the split is re-derived; the fold set above is the one that produced the numbers in Section 4.

On each fold the feature model (codebook, standardization, and PCA if any) is fit on the other training images, including their horizontal flips. Flips are not placed in the held-out set. The held-out places are scored with balanced accuracy and with both recalls.

3.6 Locked pipeline and how to rerun it

After Stage B, the winning pair is refit on all training images plus horizontal flips. It is saved with joblib, and the validation split is scored once for the report.

The test set is scored once on the repaired split, and the per-image predictions of that single evaluation are what Section 4.4 and Section 5 are computed from. The fit was chosen on the location holdout and never on the test set, so the selection rule was not informed by these 60 predictions. What limits the reading is that the test places are not all untouched: the repair of Section 2.7 was found by re-reading frames grouped by physical place, and their per-place rates had been computed before that repair. The numbers in Section 4 are therefore a corrected evaluation of the collection, not a wholly independent blind test.

Commands, from the repository root. The environment is Python 3.12 with the pins in `requirements.txt`.

```
python -m src.task3.run_stage_a
python -m src.tools.run_stage_a_nb
python -m src.task3.run_stage_b
python -m src.task3.evaluate
```

`src.tools.run_stage_a_nb` is the supplementary pass that produces the naive Bayes columns of the table in Section 3.4. It is not part of the selection: the feature ranking is computed from the linear SVM column of Stage A alone.

Pinned packages in `requirements.txt`:

```
numpy==2.4.3
Pillow==12.1.1
opencv-contrib-python==5.0.0.93
```

```
scikit-image==0.26.0
scikit-learn==1.8.0
joblib==1.5.3
```

`opencv-contrib-python` is required. The non-contrib wheel has no SIFT, and the feature module exits rather than switching descriptors.

What the saved pipeline actually contains, read back from `models/task3_pipeline.joblib`:

Item	Value
Training images before augmentation	405 (211 outside, 194 inside) in 10 location groups
Rows the forest was fitted on	810: the 405 training images and their 405 horizontal flips
Forest	300 trees, `max_depth = 12`, `max_features = "sqrt"`, `random_state = 42`, `n_jobs = -1`
Feature vector fed to the forest	8100 dimensions: the HOG block as computed, with no separate scaler. The saved pipeline has two steps, `features` then `clf`
Saved object	`models/task3_pipeline.joblib`, 2 936 221 bytes (2.80 MiB), two steps: `features` then `clf`
Inference cost	0.68 s per photograph, measured on 20 test frames on the machine used: file read, crop, HOG, `predict` and `predict_proba` (Section 6.4)

The row count follows from the data: the training split holds 405 images, augmentation doubles it, and the fitted forest reports 810 samples. Validation and test images are never flipped, so their 69 and 60 images are scored as they are.

The three pipeline commands and the supplementary naive Bayes pass were run in that order on 2026-10-27, on the repaired split of Section 2.7, from the code in the archive, and the numbers in Section 4 are the ones that run produced. The stamp beside the results matches the shipped source, so a rerun resumes instead of prompting for `--force`. Every number in Section 4 is printed in full here as well.

Code layout: `src/task3/config.py`, `data.py`, `preprocess.py`, `features.py`, `models.py`, `cv.py`, `run_stage_a.py`, `run_stage_b.py`, `evaluate.py`. Dataset construction is `src/tools/build_dataset.py`; the supplementary naive Bayes pass is `src/tools/run_stage_a_nb.py`; the dataset and annotation helpers are `src/tools/collapse_to_venues.py` and `src/tools/apply_naming.py`. The repair of the location groups is `src/tools/recut_locations.py` (Section 2.7). The figures are shipped as images under `docs/figures/`; the scripts that drew them are working-repository tools and are not part of this archive, because none of them touches the classifier. File names are resolved only in `data.py`, to find a path. `transform` receives arrays. The local window is `src/task5/app.py` and `src/task5/predict.py` (Section 6). It loads the saved pipeline and does not fit a new one.

4. Experimental results

Selection uses location-holdout balanced accuracy only. The validation column is display. The figures in this section are drawn from the locked counts, and every number in the tables below is printed in full in this section.

The comparison is a set of configurations rather than an exhaustive search. Stage one fixes the classifier at the linear SVM so that the feature axis is isolated; stage two varies the classifier only on the two features that advanced. Every classifier was run at one fixed setting, with no parameter search anywhere in the protocol.

4.1 Stage A: linear SVM, six features

Feature	Location-holdout BA	Inside recall	Outside recall	Validation accuracy
F4 HOG + LBP	0.708	0.676	0.739	0.841
F2 HOG	0.701	0.665	0.737	0.797
F3 LBP	0.607	0.544	0.671	0.826
F6 bag of words + pyramid	0.544	0.452	0.635	0.681
F1 color histogram	0.537	0.511	0.564	0.725
F5 bag of words	0.531	0.461	0.601	0.667

F4 and F2 enter Stage B.

The per-fold balanced accuracies behind those means are the five fold columns of the table below, folded as in Section 3.5.

Feature	fold 0	fold 1	fold 2	fold 3	fold 4	mean
F1 HSV histogram	0.600	0.350	0.572	0.704	0.460	0.537
F2 HOG	0.790	0.760	0.527	0.587	0.840	0.701
F3 LBP	0.670	0.470	0.705	0.563	0.630	0.607
F4 HOG + LBP	0.810	0.750	0.531	0.607	0.840	0.708
F5 bag of words	0.580	0.600	0.527	0.558	0.390	0.531
F6 bag of words + pyramid	0.610	0.610	0.499	0.529	0.470	0.544

The color histogram is near the bottom, 0.537 against 0.708 for the winner. That matches the reason HKU was collected: red brick is on both sides of the label, and color does not survive a change of place. Bag of words is last, 0.531, and the spatial pyramid lifts it to 0.544, so the finer

spatial layout buys 0.013, a small gain and still far below the two features that advanced. The validation column and the holdout column do not pick the same pair: validation puts F3, the LBP texture, second at 0.826, while the holdout puts F2, the HOG shape, second at 0.701. Had validation been the selection score, F3 would have gone to Stage B and F2 would not. Validation accuracy is the wrong column to maximize here, and it is shown so that the disagreement stays visible.

4.2 Stage B: F4 and F2 across classifiers

C6 does not apply to F2 or F4. The winner is not a linear SVM, so the RBF row was not run.

Feature	Clf	Location-holdout BA	Inside recall	Outside recall	Validation accuracy
F2	C4 random forest	**0.722**	0.622	0.821	0.783
F4	C4 random forest	0.712	0.627	0.797	0.841
F4	C2 logistic regression	0.712	0.660	0.764	0.826
F4	C1 linear SVM	0.708	0.676	0.739	0.841
F2	C2 logistic regression	0.706	0.660	0.751	0.797
F4	C3 k-NN	0.703	0.562	0.845	0.826
F2	C1 linear SVM	0.701	0.665	0.737	0.797
F2	C3 k-NN	0.689	0.527	0.851	0.754
F4	C5 histogram gradient boosting	0.679	0.606	0.752	0.899
F2	C5 histogram gradient boosting	0.674	0.593	0.756	0.812

The locked pipeline is **F2 HOG + C4 random forest**.

On the same F2 features, the linear SVM's validation accuracy is 0.797 and the forest's is 0.783, and the forest is the one selected because its location-holdout balanced accuracy is higher (0.722 against 0.701). k-NN's outside recall is higher (0.851 on F2) and its inside recall is 0.527. On fold 3, which holds out IN-04 and OUT-04, k-NN scores 0.471, the lowest of the five, while the forest scores 0.638, the highest.

The five folds of the location holdout are the evidence behind that mean. They are the folds of Section 3.5: IN-01 and OUT-01, IN-02 and OUT-02, IN-03 and OUT-03, IN-04 and OUT-04, and IN-01 together with OUT-05 and OUT-08.

F4 classifier	fold 0	fold 1	fold 2	fold 3	fold 4	mean
C1 linear SVM	0.810	0.750	0.531	0.607	0.840	0.708
C2 logistic regression	0.810	0.760	0.562	0.587	0.840	0.712

F4 classifier	fold 0	fold 1	fold 2	fold 3	fold 4	mean
C3 k-NN, k=5	0.800	0.730	0.578	0.498	0.910	0.703
C4 random forest	0.740	0.740	**0.567**	0.646	0.870	0.712
C5 histogram gradient boosting	0.750	0.730	0.528	0.567	0.820	0.679

The forest does not win every fold. On F4 it is last on fold 0 (0.740 against 0.810 for both linear models) and second on fold 4 behind k-NN. k-NN has the opposite shape: it wins fold 4 and fold 2 but is last on fold 3 at 0.498. What the mean rewards is the middle of the distribution, and the forest has the highest floor: its worst fold is 0.567, and every other classifier drops below that somewhere, k-NN to 0.498, gradient boosting to 0.528, the linear SVM to 0.531, logistic regression to 0.562. Two consequences follow. The forest has the highest mean score and, in this comparison, also the highest minimum fold score; but a ranking by mean is not a ranking by floor in general, and the selection rule ranks by mean. And the spread within one classifier, 0.498 to 0.910 for k-NN, is wider than the whole span of the five means, 0.679 to 0.712.

Gradient boosting never places first on any fold, which is why its mean, 0.679, is the lowest of the five.

The same five folds on the second feature give the same evidence for F2:

Feature	Clf	fold 0	fold 1	fold 2	fold 3	fold 4	mean
F2	C1 linear SVM	0.790	0.760	0.527	0.587	0.840	0.701
F2	C2 logistic regression	0.780	0.790	0.531	0.577	0.850	0.706
F2	C3 k-NN, k=5	0.800	0.770	0.574	0.471	0.830	0.689
F2	C4 random forest	0.750	0.800	0.560	0.638	0.860	0.722
F2	C5 histogram gradient boosting	0.740	0.760	0.479	0.584	0.810	0.674

The forest wins on F2 as well, 0.722 against 0.706 for logistic regression and 0.701 for the linear SVM. On fold 2, which holds out IN-03 and OUT-03, all ten feature--classifier pairs fall between 0.479 and 0.578, so on that fold the held-out place moves the score more than either axis of the comparison does.

F2 logistic regression has a higher holdout inside recall than the forest (0.660 against 0.622) and a lower balanced accuracy (0.706 against 0.722). The written score is balanced accuracy. The test set's later inside recall does not reopen that choice.

F4 random forest and F4 logistic regression both print as 0.712, which is a rounding artefact and not a tie. The unrounded means are 0.712456 for the forest and 0.711810 for logistic regression,

so the forest is the marginally stronger of the two. Neither reaches F2 with the forest at 0.722, so the winner is unchanged.

The representation moves the score more than the classifier does: the span within one feature set is 0.047 on F2, from 0.674 to 0.722, and the move from colour to HOG under one linear SVM is 0.164, from 0.537 to 0.701.

One seed was used, 42, and the folds are not independent: IN-01 is held out in two of the five folds, so there are effectively four independent inside training places. The fold-to-fold spread is far wider than the gaps between the top features or between the top classifiers. F4 leads F2 by 0.007 under the linear SVM, and the forest leads the linear SVM on F2 by 0.021, while the selected configuration alone moves from 0.560 on fold 2 to 0.860 on fold 4. Small differences between configurations should not be read as a real ordering, and the margin that decided the locked pair, 0.009 between F2 and F4 with the forest from the unrounded means, is inside that spread.

4.3 Validation after the refit

The forest was refit on the full training split, with horizontal flips, and then applied to the 69 validation images (IN-06 40, OUT-09 29).

Accuracy **0.783**. Balanced accuracy **0.784**.

	Predicted outside	Predicted inside
True outside	23	6
True inside	9	31

Class	Precision	Recall	F1	Support
outside	0.719	0.793	0.754	29
inside	0.838	0.775	0.805	40

These three-decimal figures are the published rounding of the matrix: $54/69 = 0.783$, outside recall 23/29 rounds to 0.793, inside recall 31/40 rounds to 0.775. Nine campus frames are called outside and six harbourfront frames are called inside. Figure 4 draws the same four cells, and each cell also carries its share of the true-class row. Section 2.5 is why 0.783 is not quoted below as generalization.

4.4 Test, one evaluation on the repaired split

60 images: IN-05 16, IN-07 17, OUT-06 20, OUT-07 7.

Accuracy **0.733**. Balanced accuracy **0.754**.

	Predicted outside	Predicted inside
True outside	26	1
True inside	15	18

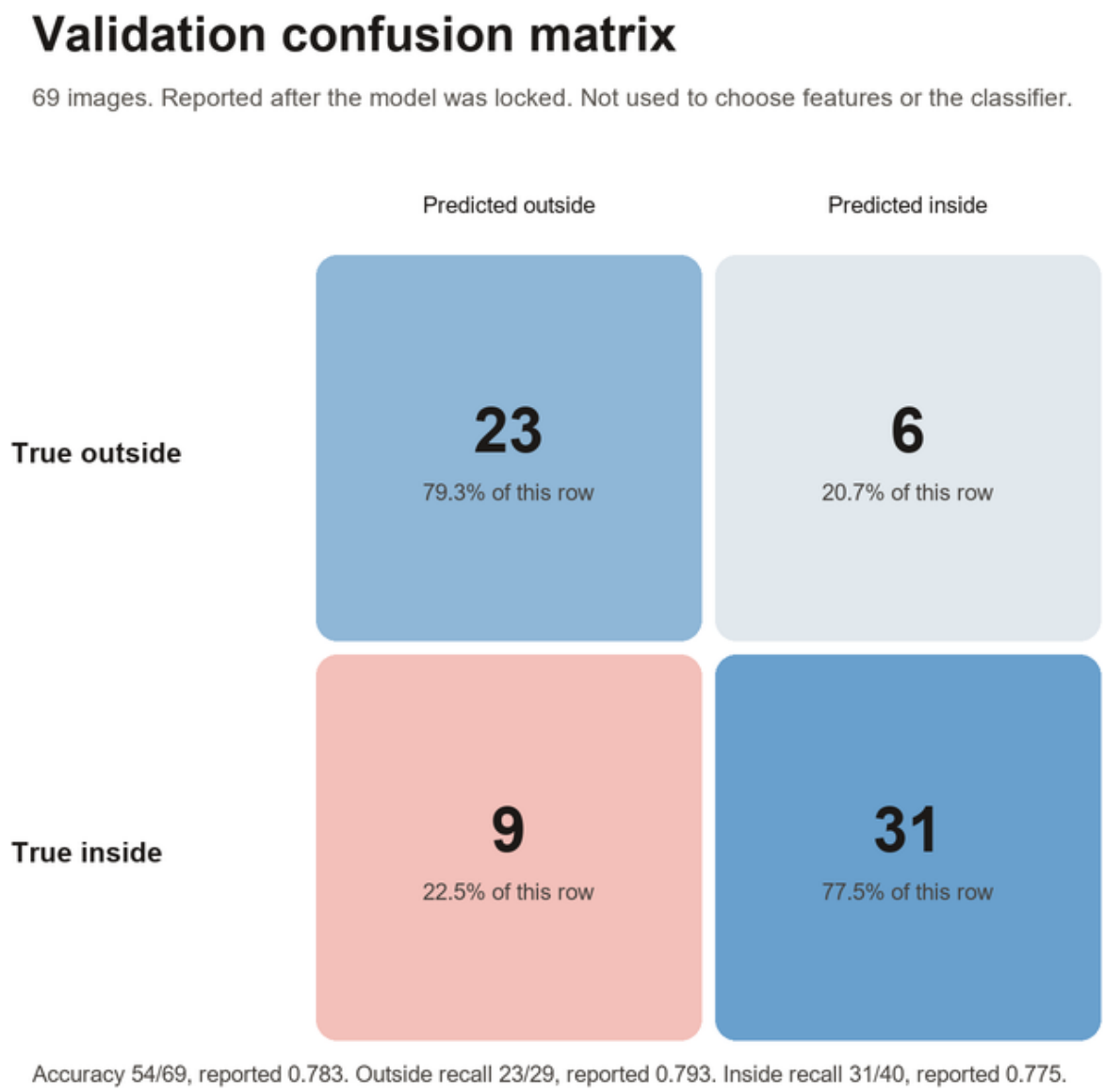
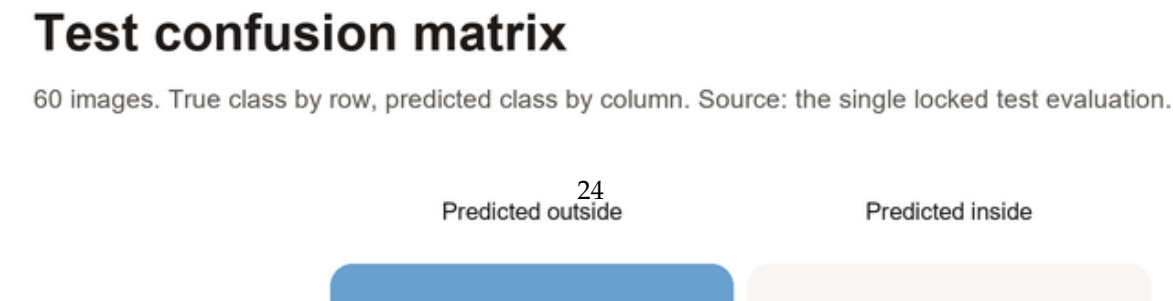


Figure 4. Validation confusion matrix of the locked pipeline. The four counts are the ones in the table above. Not used to choose the model.

Class	Precision	Recall	F1	Support
outside	0.634	0.963	0.765	27
inside	0.947	0.545	0.692	33

Working from the same cells: accuracy 44/60 rounds to 0.733 and balanced accuracy to 0.754; the recalls are 26/27 = 0.963 outside and 18/33 = 0.545 inside, with precisions 26/41 = 0.634 and 18/19 = 0.947. Figure 5 draws the same four cells the same way. The same matrix recomputes from the per-image predictions of that evaluation (outside 0.6341 / 0.9630 / 0.7647, inside 0.9474 / 0.5455 / 0.6923).



Predictor	accuracy	balanced accuracy	outside recall	inside recall
Always outside	27/60 = 0.450	0.500	1.000	0.000
Always inside	33/60 = 0.550	0.500	0.000	1.000
Locked pipeline	44/60 = **0.733**	**0.754**	0.963	0.545

The test split holds 33 inside images and 27 outside ones, because the split unit is the place and the two held-out inside places are larger than the two held-out outside places. Both constants are beaten on both metrics: the always-inside constant scores 0.550 on accuracy and the always-outside constant 0.450, while the model scores 0.733; on balanced accuracy the model is 0.254 above either. The margin over the stronger constant is therefore 0.183, which is not a thin margin. The model keeps 0.963 outside recall where the always-inside constant has 0.000, and when it does say inside it is right 18 times out of 19.

The two errors are not symmetric for an application. There is one false inside in the whole test set, the OUT-06 frame that shows the Hong Kong University Main Building. There are 15 missed campus photographs, and Section 5.2 shows that 12 of the 17 frames of the CMA courtyard are among them. A user who must not miss campus scenes would be hurt by those 15, and the balanced accuracy of 0.754 is the number that carries that cost; the accuracy of 0.733 does not.

One weakness of the evidence belongs in the other direction. The selection score was fixed before the run and it was balanced accuracy, and the margin between the locked configuration and the runner-up is 0.009 from the unrounded means, well inside the fold-to-fold spread that Section 4.2 records. The test set also rests on four places, one of which, IN-07, carries 17 of the 60 images and accounts for 12 of the 15 misses. A different fold design or a different pair of held-out places would move this number, and the report quotes one evaluation of one pipeline rather than a range.

---

5. Error analysis

This section reads the test matrix place by place, and then reads the frames themselves.

5.1 How to read the test matrix

The four cells are not one story. 26 of the 27 outside images are correct and 1 is called inside, which is the only false inside in the test set; on the other side 15 of the 33 inside images are called outside, and that is the failure. Outside precision is 0.634 because the 15 campus misses sit in the same predicted-outside column as the 26 true outside images, and inside precision is 0.947 because the inside column is almost clean. Accuracy 0.733 hides that split. Balanced accuracy 0.754 sits above the 0.722 that the same configuration scored on the location holdout, which says the four held-out places happen to be easier for it than the average held-out training place; it does not say the pipeline is stronger than the selection score suggested.

5.2 Failure by place

The test predictions were split by group. Each group has one label, so a group's errors can land in only one off-diagonal cell.

Group	Correct / n	Accuracy	Where the errors go
OUT-06 HKU Main Building	19/20	0.950	1 image predicted inside. This is the only false inside in the matrix.
OUT-07 HKU campus green	7/7	1.000	None.
IN-05 Main entrance forecourt	13/16	0.812	3 images predicted outside.
IN-07 CMA courtyard	5/17	0.294	12 images predicted outside.

Check: 19+7+13+5 = 44, the numerator of 44/60. The 3 and 12 misses sum to the 15 false outsides, and the single OUT-06 miss is the single false inside, so OUT-07 cannot contain that false inside. Figure 6 plots these four accuracies against the overall test accuracy.

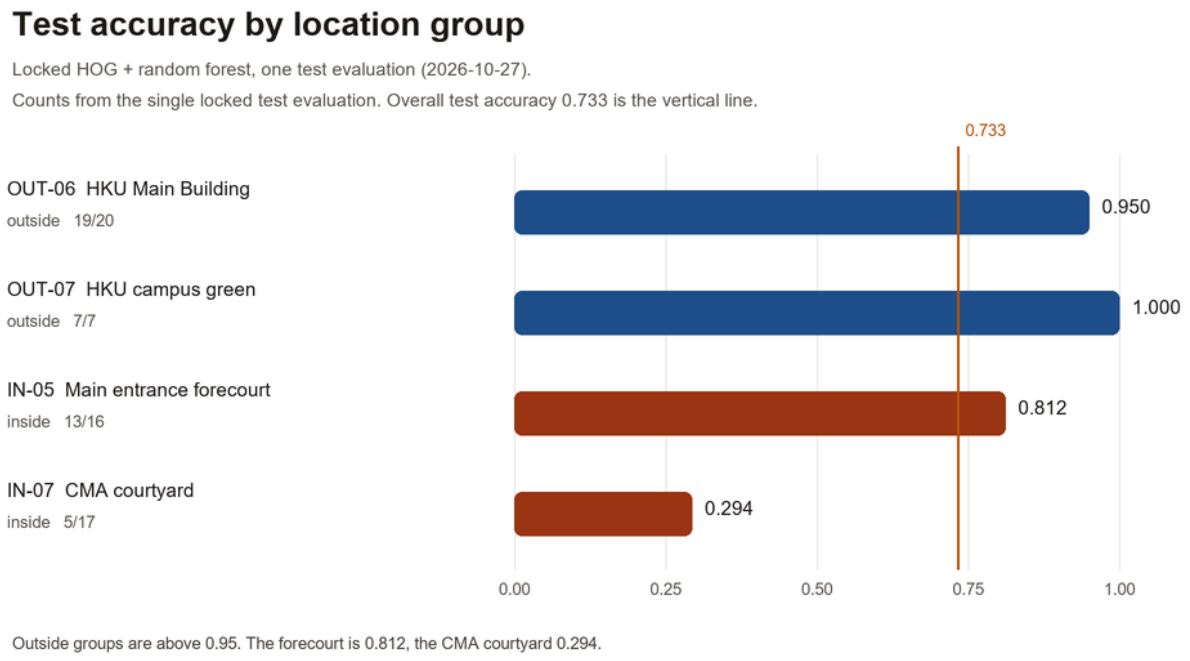


Figure 6. Test accuracy of the four held-out places. The vertical line is the overall test accuracy 0.733.

Five things hold across the four held-out places:

1. The two unseen campus places are not equally hard. The forecourt is 13/16 and the CMA courtyard is 5/17, and the courtyard carries 12 of the 15 misses on its own.
2. The two unseen outside places are almost all predicted outside, 19/20 and 7/7.
3. The red-brick hard negative is not accepted as PolyU: the Hong Kong University Main Building is classified outside on 19 of its 20 frames. Why that frame is rejected is not settled by these numbers.
4. The forecourt is the campus place the pipeline fails on least, 3 of its 16 frames, and it is also the one whose frames most often fill the crop with the red-brick gate building, the same material as the brick negative. Section 5.4 reads those frames one by one.

5. The green pair does not transfer either. OUT-07, a tree-lined lawn, is 7/7 outside. IN-07, whose 17 frames are 7 green space, 4 building entrance, 4 façade and 2 tree-lined path, is 5/17.

On the final refit, inside recall is 0.775 on validation and 0.545 on test. During selection, the same feature and forest had a mean inside recall of 0.622 across the five training-location folds. Those three numbers use different fits. They are not epochs of one curve. They do show the same direction: campus scenes that were not in the fit are the ones dropped.

On the unseen campus places the model leans towards outside: 15 of the 33 campus images are called outside. It is not the constant, which would call all 33 outside, and on balanced accuracy it is 0.254 above either constant. A constant would leave the 18 correct campus frames unexplained, and the 0.254 gap in balanced accuracy is what separates the two.

5.3 Relation to HOG

Adding LBP to HOG does not help on this data. Under the linear SVM it moves location-holdout balanced accuracy from 0.701 to 0.708, and under the forest it moves from 0.722 to 0.712 on the printed values, which is $+0.007$ and -0.009 computed from the unrounded means. Both moves are far inside the fold-to-fold spread. The locked feature is the HOG block alone, and the failure was measured with that representation. Which property of the representation is responsible is not settled by these numbers.

HOG counts edge orientation inside 16×16 cells. It does not encode which building this is, or which side of the campus boundary the camera stands on. Color is discarded. Most of the building's layout is discarded. A forest of depth 12 can split training places whose edge patterns it has seen. The training campus places are the J--VA walkway, the P/Q coffered walkway, the Library and the E block, and FSCH / the pool. The test campus places are the main entrance forecourt and the CMA courtyard. Those edge statistics do not have to match.

Two representations that might seem more semantic were measured on the location holdout and were worse:

- Color is the cue most tempted by red brick. It scored 0.537, fifth of the six in Stage A. It was not evaluated again on the test set, so no test figure for it exists. The test behaviour of the locked model is still informative: OUT-06 is rejected (19/20) while the forecourt is also rejected on 3 of its 16 frames and the CMA courtyard on 12 of its 17. Colour was not selected, on a holdout score of 0.537; whether dropping it helped on these test places was not measured. What remains cannot say ``this brick building is the campus''.
- Bag of words scored 0.531, last, and the spatial pyramid lifts it to 0.544. A codebook is refit inside every fold, so at scoring time it never held the training buildings, and a keypoint histogram is the representation most exposed to a change of place.

One mechanism is consistent with the feature definition: a new campus layout can fall into parts of the HOG space where the forest has seen mostly outside training places. That was not measured directly; the forest's splits were not probed.

Five explanations are worth separating before the misses are attributed to anything: background bias, lighting, similar places, viewpoint, and road-frontage imbalance.

Candidate explanation	What is measured here
Similar-looking places	Measured. OUT-06 against the forecourt (brick), OUT-07 against IN-07 (green / court). Outside is recognized. The forecourt is 0.812 and the courtyard 0.294.
Background bias	The scene table was built so that 11 of 13 types exist in both classes. Test errors cluster by place: 12 of the 15 misses are in one place. That does not support a story in which one background type produces all the misses.
Lighting	Night is 10.2% of the usable inside pool and 13.6% of the outside pool. All thirty outside night frames are in training; the seven inside night frames in test are all from IN-05, the best-scoring test inside place at 13 of 16. The prediction table carries no day/night field, so a night-error rate is reported as not applicable rather than estimated. The misses are concentrated in IN-07, which has no night frame at all, so the night share does not explain them. Night is confounded with place rather than balanced across it, and no colour jitter was applied, so the axis was collected and not diluted.
Viewpoint	Viewpoints vary inside each place by design, and the error table has no viewpoint column, so no viewpoint-error rate is computed. The qualitative set does show that distance is not what separates the correct CMA frames from the missed ones: a close view of the patterned wall is correct and a wider view of the same wall is not, and a wide courtyard view is correct while another wide courtyard view is not. What the crop contains, rather than how far away the camera stood, is what Section 5.4 reads frame by frame.
Road-frontage imbalance	10 inside versus 57 outside in the usable pool. The test places are not that road set. The misses are not attributed to traffic.

This is the score of this HOG pipeline on places it has not seen, under a protocol that refuses to train on those places. It is not a bound on the feature: one configuration, one seed and one fold design were measured, and no variant of the feature or of the classifier was tried. The number is final for this pipeline; it is not a grid that stopped halfway.

5.4 Qualitative examples

Figure 7 is the contact sheet of individual test predictions, and it follows this discussion. The collection map is Figure 2 in Section 2.1.

The sheet was assembled from the locked per-image test predictions and the photographs in `data/pictures/`, the directory the dataset archive from `DATASET_LINK.txt` unpacks into. Its four quadrants are exactly the four cells of the test matrix in Section 4.4, 26, 1, 15 and 18, and the place counts are the ones Section 5 reports, 19/20, 7/7, 13/16 and 5/17, so the sheet cannot show a frame the tables do not count. `docs/figures/task4_example_manifest.csv` records which file sits in which cell.

The thirteen frames were chosen by a rule fixed in advance, not by how they look: within each cell, in filename order --- 2 correct OUT-06, 2 correct OUT-07, the single false inside (necessarily OUT-06), 2 missed IN-05, 2 missed IN-07, 2 correct IN-05, and 2 correct IN-07. The rule ran before any frame was looked at, so no cell could be filled by choosing an example that suited the argument.

The grid shows the two failure modes at once. The one false inside is `OUT-06_012_IMG_0892.jpg`, a glass entrance and a flight of steps at HKU, not the red-brick facade that dominates its group; the other 19 frames of that group are correct.

The missed inside frames are `IN-05_024_IMG_1327.jpg` and `IN-05_025_IMG_1328.jpg`, the two frames at the main entrance in which the crop is filled by the red-brick name-wall building with the flags in front of it, and two frames at the CMA courtyard, `IN-07_006_IMG_1432.jpg` and `IN-07_007_IMG_1433.jpg`, which are the brick building and its planting. Ten further CMA frames are missed as well, but the sampling rule takes the first two in filename order, so only those two are in the grid. The two correct forecourt frames are `IN-05_012_IMG_1315.jpg` and `IN-05_013_IMG_1316.jpg`, and the two correct CMA frames are `IN-07_001_IMG_1427.jpg` and `IN-07_002_IMG_1428.jpg`.

The crops are what the model sees, and no single cue separates the four forecourt frames. What can be said is what each crop contains. The two correct forecourt frames are the wider views: the flags stand in the middle of an open paved plaza with a lawn beyond, and the brick building is small in the frame. The two missed ones are the close views in which the same building fills the upper half of the crop, so the frame is carried by brick, flagpoles and a hard edge of shadow rather than by the shape of the forecourt. At the courtyard the split runs the other way: `IN-07_002` keeps the PolyU crest on the wall of the brick building, `IN-07_001` keeps a white canopy with the planters under it, and the two missed frames are the brick building and its planting without either mark.

That is thirteen frames read one by one, and it is an example set rather than a statistic. What it supports is narrow: the misses are the frames in which the identifying mark of the place is absent from the crop or replaced by something adjacent --- a wall of brick instead of a forecourt, a building instead of its crest --- and the one false inside is the frame in which the glass entrance of an outside building looks like a campus entrance. What it does not support is a claim that brick as such is the cause, because a correctly classified courtyard frame is brick as well.

Those results are consistent with a limit of the representation, and also with a limit of how much of each place the collection covers: the two were not separated, and no experiment in this report distinguishes them. Nothing here rules out the collection side either. A crop that cuts the identifying mark out of a place, and a place whose frames never show such a mark,

would both produce this pattern, and the report did not test which of the two is at work. HOG describes local gradient, so a crop made of brick ribs, paving and café furniture is a gradient field like any other; a crest or a sign reaches the descriptor only as edges and local contrast, and whether a particular arrangement of them is enough to identify a place was not tested. Hong Kong University's red brick was put in the outside class deliberately, in OUT-02 (train) and in the eleven Chinese-garden frames of OUT-03 (train), so that brick would not be a free cue. The Main Building itself, OUT-06, was kept in the test set and was never trained on; its 95% accuracy is test evidence and says nothing about what the model learned. The report cannot separate viewpoint from material here: the prediction table has no viewpoint column, so no viewpoint-error rate is computed (Section 5.3). What can be said is narrow --- the wide view of the forecourt is correct while the close view of the same building is not --- and class balance was not ruled out either, since the test split holds 33 inside images against 27 outside ones.

5.5 The test numbers do not retune the model

Accuracy 0.733 and inside recall 0.545 are the score of the pipeline that was locked on location-holdout balanced accuracy. IN-05 and IN-07 stay in the test set. Features are not re-ranked on these 60 images. The forest is not swapped for logistic regression because logistic regression had a higher holdout inside recall on F4. A later check of whether more campus places would change the picture needs new places and a new held-out evaluation of this same pipeline. It does not move the current test groups into training.

---

6. Application

The local program lets a user select one scene photograph, shows that photograph, and returns either ``Inside PolyU main campus'' or ``Outside PolyU main campus''. It is a desktop window written with Tk from the Python standard library. Nothing was added to `requirements.txt`. Pillow, joblib, OpenCV [9], scikit-image [8] and scikit-learn [7] are the training dependencies. The checks are listed in Section 6.4 and the runs in Section 6.5. The same launch lines are in the root `README.md`.

The program does not train and does not edit `splits.csv`. The classifier remains the locked HOG random forest in `models/task3_pipeline.joblib`. The test accuracy 0.733 and the other Section 4--5 figures are unchanged.

6.1 How to launch

From the repository root, in the same environment as the training code:

```
python -m src.task5.app
```

That opens the window. With no display, the same loader and the same preprocessing print one image and do not open a window:

```
python -m src.task5.app --image path\to\photo.jpg
```

The command prints the English label and, for this forest, the outside and inside probabilities. If the file cannot be read, it prints the error and exits with status 1.

On Windows, `pythonw -m src.task5.app` opens the window without a console. The official Windows installer includes Tk. If Ubuntu reports that `_tkinter` is missing, install `python3-tk`. If `models/task3_pipeline.joblib` is absent, the window tells the user to run `python -`

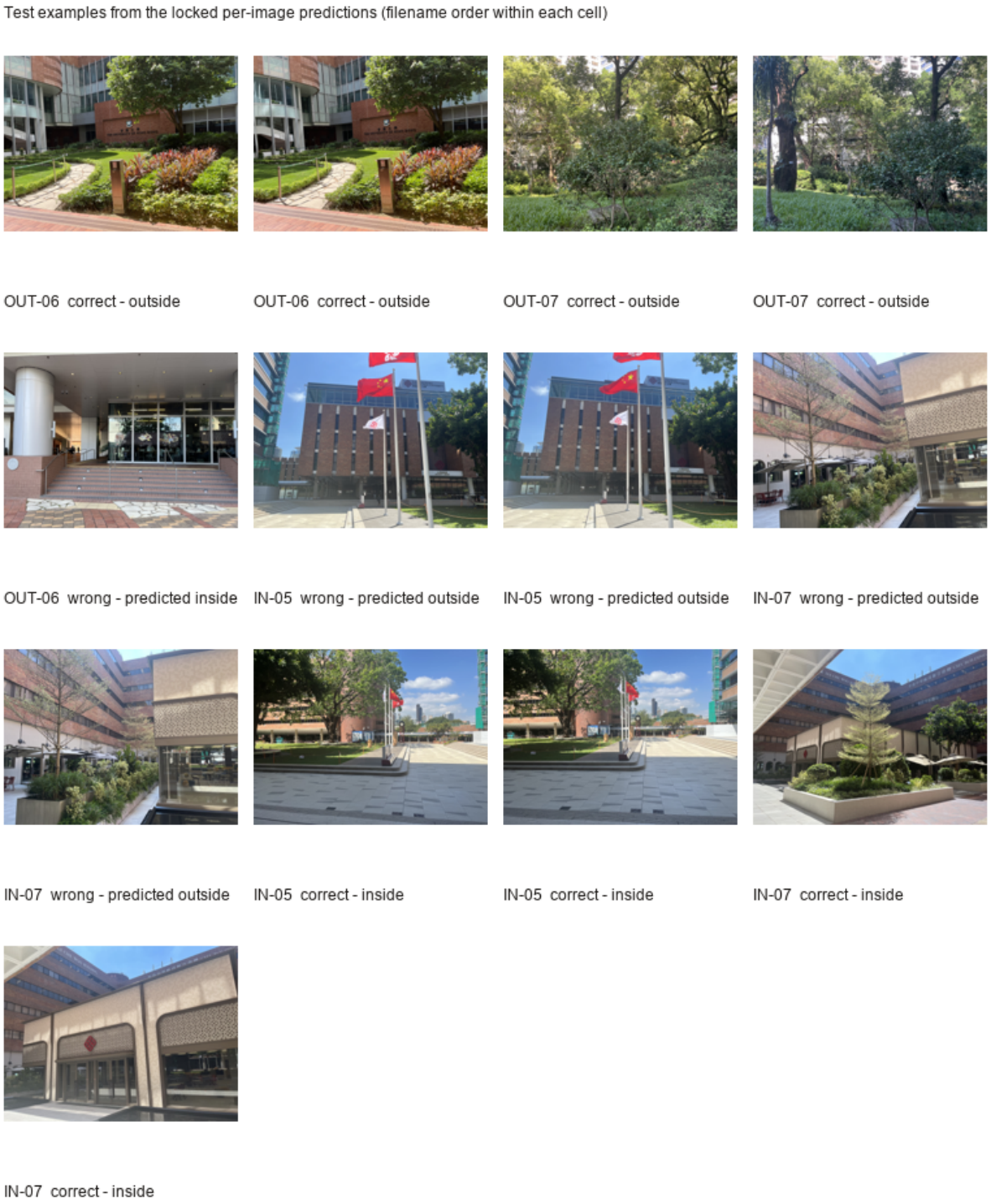


Figure 7. Thirteen test photographs in the order of the manifest: each caption gives the location group and whether the locked pipeline classified that frame correctly. The file name of every frame is in `docs/figures/task4_example_manifest.csv`.

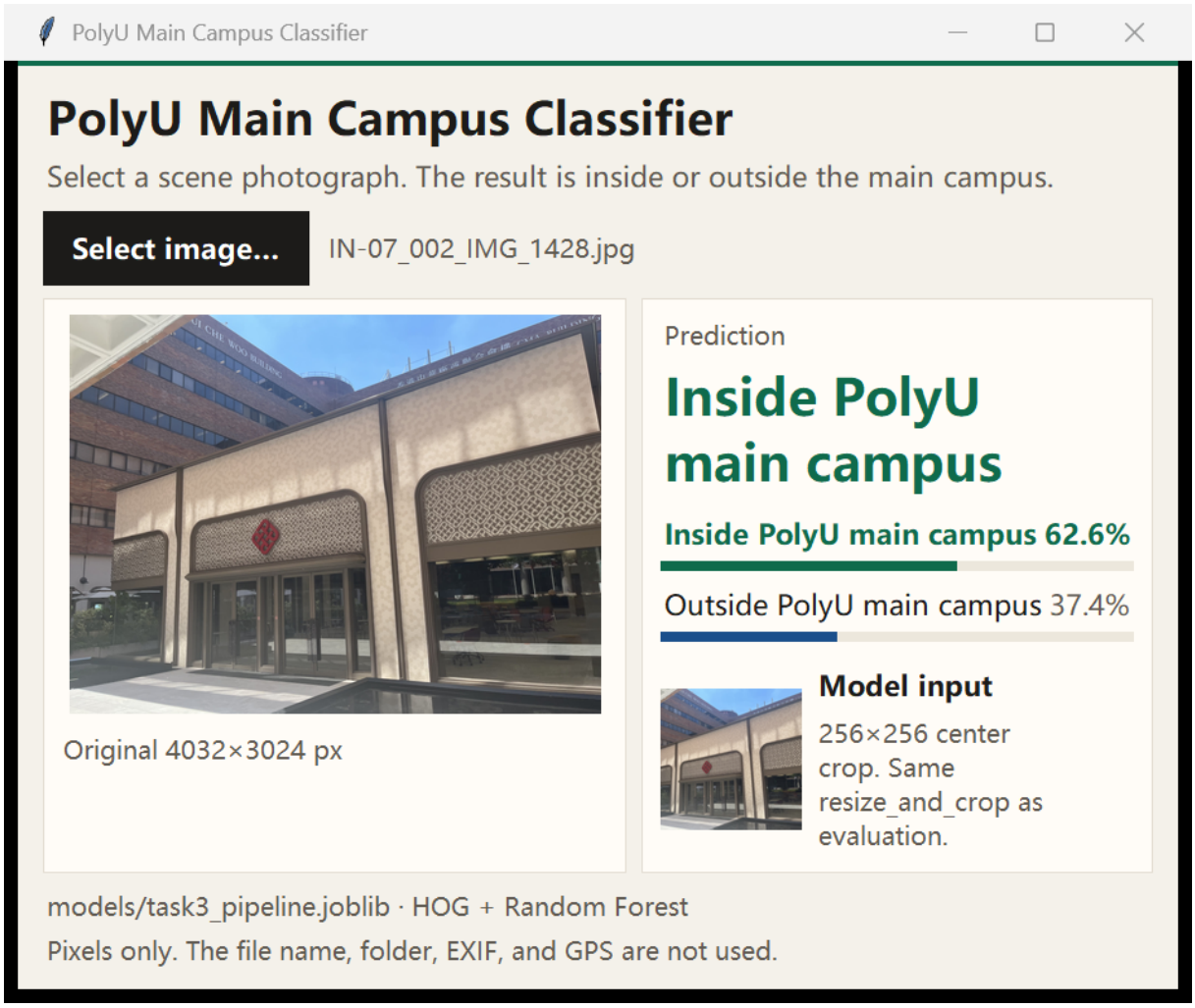


Figure 8. The application running with a campus photograph loaded. This is a screenshot of the running program. The photograph is `IN-07_002_IMG_1428.jpg`, the CMA courtyard with the PolyU crest on the wall, which the locked pipeline calls inside with 62.6%. The left pane is the photograph as the user selected it, the right pane is the sentence and the two class probabilities the program returned, and the small image is the 256×256 centre crop the model actually receives. The bottom line names the pipeline file and states that the file name, folder, EXIF and GPS are not used.

`m src.task3.evaluate` and does not run that training step itself. In this submission the saved file is already present.

6.2 What the window shows

The window title is ``PolyU Main Campus Classifier''.

1. **Select image...** opens a file dialog. Ctrl+O does the same. The dialog accepts jpg, jpeg, png, bmp, tif, tiff, and webp.
2. The left pane shows the selected photograph and its original width and height. The file name is written next to the button so the user can see which file is open. That string is not given to the classifier.
3. The right pane prints the sentence stored on the pipeline as `display_`: ``Inside PolyU main campus'' or ``Outside PolyU main campus''.
4. This random forest implements `predict_proba`, so the window also draws the inside probability and the outside probability.

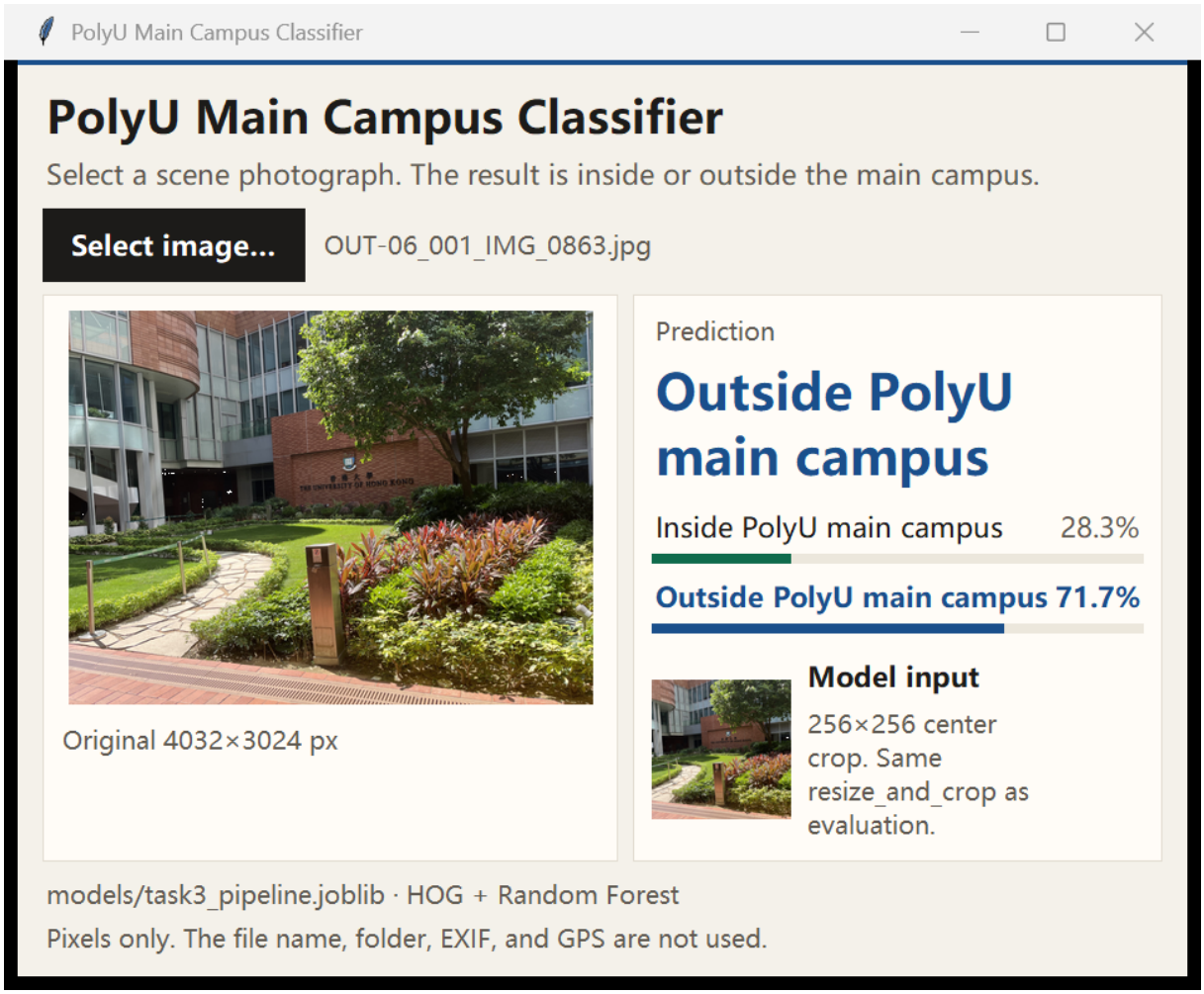


Figure 9. The same application with an outside photograph loaded. The photograph is `OUT-06_001_IMG_0863.jpg`, the University of Hong Kong Main Building with its sign on the wall, which the locked pipeline calls outside with 71.7%.

5. The thumbnail is the array returned by `resize_and_crop`: the 256×256 center crop that the model receives. The widget scales that crop for display. The array passed to `predict` stays 256×256×3.

A line at the bottom of the window names `models/task3_pipeline.joblib` and states that the file name, folder, EXIF, and GPS are not used.

6.3 How prediction is called

`src/task5/predict.py` is the prediction path. `src/task5/app.py` is the window and the `--image` printer. The order matches `src/task3/evaluate.py`: crop, then predict.

1. `load_rgb` in `src/task3/preprocess.py` reads the file as RGB `uint8`. It does not rotate from EXIF. Training and evaluation did not rotate, so the application does not add a rotation the model never saw.
2. `resize_and_crop` in the same module scales the short side to 256 and center-crops to 256×256×3. There is no second resizer.
3. `pipeline.predict([cropped])` returns 0 for outside or 1 for inside. The argument is a one-image list. A bare array would be iterated by rows.

4. `pipeline.predict_proba([cropped])` fills the two probabilities. If a future estimator had no `predict_proba`, the window would say so and would still show the label. This forest has the method.
5. `pipeline.display_[label_id]` is the English sentence written when the pipeline was saved: `{0: "Outside PolyU main campus", 1: "Inside PolyU main campus"}`.

`joblib.load` returns a scikit-learn [7] `Pipeline` with two steps. `features` is `ImageFeatures` with `feature_id="F2"`, whose inner feature object is `HogFeat`; the two are read back from the saved file, not assumed. `clf` is `RandomForestClassifier` with 300 trees, `max_depth=12`, and `random_state=42`. `classes_` is `[0, 1]`.

`HogFeat.transform` calls `resize_and_crop` again. When the image is already 256×256, that function returns the same array. The call written in the application is still `src.task3.preprocess.resize_and_crop`.

`predict_rgb` accepts an RGB array. The path is used only to read those pixels and to label the button row with the file name.

6.4 Checks recorded for the application

These checks were run on the installed `requirements.txt`. They do not refit the forest and they do not replace Section 4 or Section 5.

- From open to label, one image took 0.68 s on average over 20 test frames on the machine used, including the file read, the crop, HOG, `predict` and `predict_proba`.
- Every crop is `(256, 256, 3)`. Calling `resize_and_crop` again on that crop leaves the array unchanged. Predicting the uncropped original returns the same label as cropping first.
- The same pixels saved as `IN-99_inside_campus.jpg` and as `OUT-99_outside_campus.jpg` produce the same label and the same probabilities. The name in the window changes. The sentence does not.
- An unreadable text file opens an error dialog and keeps the previous label. The command-line form exits with status 1.
- Noise, a 1×1 image, a very wide image, all black, all white, a grayscale JPEG, a PNG with an alpha channel, and a 3840×2400 JPEG each returned one of the two English sentences.
- The window was exercised by hand on one display. After a file was chosen, the canvas showed the original, the thumbnail showed the crop, and the prediction text was one of the two sentences. `python -m src.task5.app --image <file>` and `python src/task5/app.py --image <file>` printed that same label and both probabilities.

All 60 test images were replayed through the program's own prediction path, `predict_rgb` in `src/task5/predict.py`, which is the function the window calls. The label it returned matched the locked prediction table on 60 of 60 images. That is the direct check that the application and the evaluation are one pipeline: the same `models/task3_pipeline.joblib`, the same `resize_and_crop`, the same pixels, and the same answer. The replay is reproduced by running `python -m src.task5.app --image <file>` over the 60 test photographs and comparing the labels.

Two scenes from the course material were also run through the program, to confirm that it returns a label and does not crash on a photograph it was not trained on. They are the two halves of Figure 1 of the assignment, a 1920×1080 image whose printed captions give the ground truth: the left half is a main-campus courtyard near the Anita Chan Lai Ling Building, labelled inside, and the right half is a public street beside Wing On Centre in Sheung Wan, labelled outside.

Each half is 960×1080, so both are portrait frames. They are not part of the test set and were not used to change the model.

Example scene	Pixels	True label	Program output	outside	inside
Courtyard	960×1080	Inside PolyU main campus	Inside PolyU main campus	0.499	0.501
Street	960×1080	Outside PolyU main campus	Outside PolyU main campus	0.513	0.487

Both examples were classified correctly, but their predicted probabilities were close to 0.5, within 0.013 of it in all four cells. These two examples demonstrate successful inference, not reliable generalisation. What the table does show is the input path: both frames come from outside the collection, and the program still returns one of the two labels and a pair of probabilities on them, which is the behaviour Section 6.1 and Section 6.3 describe.

No accuracy is estimated from these two frames. The held-out test result remains accuracy 0.733, inside recall 0.545, and the four place rates in Section 5.

6.5 Real runs

Two runs of the program on photographs from the test split, with the output exactly as it was printed. The first is a campus photograph, the second an outside one, and both labels agree with the locked prediction table.

```
> python -m src.task5.app --image \
    data/pictures/inside/U_Garden_Area/IN-07_002_IMG_1428.jpg
Inside PolyU main campus
outside: 0.374
inside: 0.626
```

```
> python -m src.task5.app --image \
    data/pictures/outside/HKU/OUT-06_001_IMG_0863.jpg
Outside PolyU main campus
outside: 0.717
inside: 0.283
```

Figure 10 is the same prediction path drawn as a single figure, so that the three things the model works on can be read side by side: the file the user selected, the 256×256 crop the model received, and the sentence and probabilities the program returned. The photograph is `OUT-06_001_IMG_0863.jpg`, the Hong Kong University Main Building, one of the hard negatives, and the locked pipeline calls it outside.

The two screenshots in Figures 8 and 9 were taken from the running window. The photograph was loaded through the application's own `load_and_predict`, so the canvas, the crop thumbnail and the prediction line are the widgets the program draws; the one step not exercised in that run is the operating system's file dialog itself. The pixels that reached the classifier are the pixels of the file on disk, and the two numbers in the figures are the two numbers in the tran-

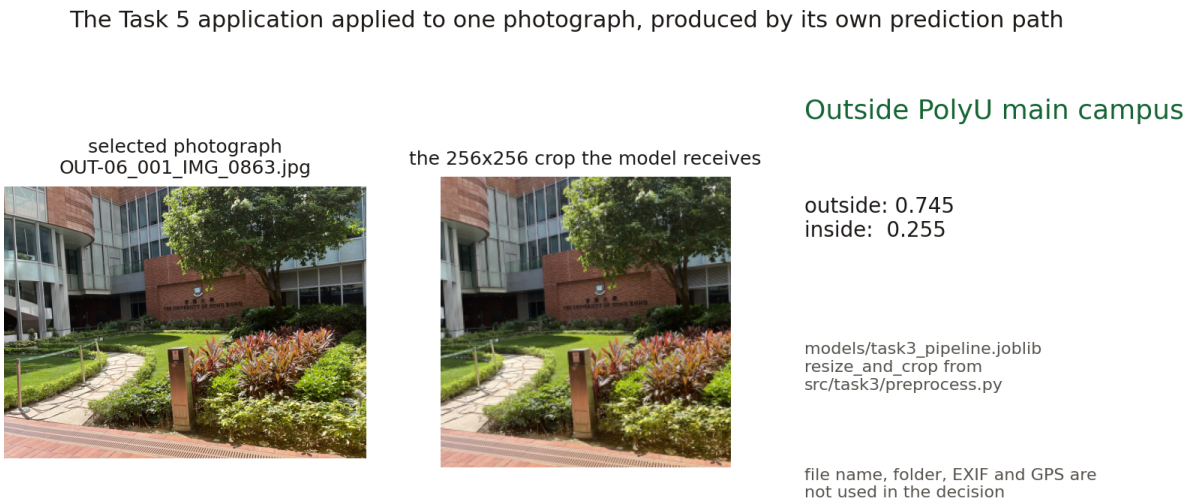


Figure 10. The prediction path applied to one photograph, drawn from the program's own output. The left panel is the selected file, the middle panel is the 256×256 centre crop the model receives, and the right panel is what the program returned: the sentence and the two class probabilities. The last two lines of that panel are the figure's own note, that the file name, folder, EXIF and GPS are not used, not output from the program.

scripts above. Both photographs are test images, so the two sentences above are also the labels in the locked prediction table.

---

7. Discussion

Six questions about how the program was designed and tested, how far its robustness reaches, what went wrong along the way, how AI was used, how it reasons about the task, and where that use stops. The answers use the dataset, the result tables of Section 4, and the application checks in Section 6.4.

How do you design and test the program?

The geographic rules came first: main campus versus not, boundary frames dropped, one physical place as the split unit. Collection then filled both classes with matched scene types and with outside places that look like campus, especially HKU brick.

The learning program was designed as two locked stages rather than a search over every feature--classifier pair: Stage A compares six pixel features with one linear SVM, Stage B compares classifiers on the two features that advanced, and the score is the mean balanced accuracy of five pre-written location-holdout folds. Section 3.1 sets out why each of those choices was made.

Testing of the data side is the check inside `build_dataset.py`: no group crosses a split, the two mutually visible West Kowloon groups stay together, and every split contains both classes. The numbers were checked separately: the published tables were matched against the Stage A and Stage B result tables value for value, and the test confusion matrix was recomputed from the per-image test predictions of the locked evaluation. Statements about feature dimensions come from reading the code; the numbers come from the stored result tables. The commands and package pins are in Section 3.6.

The desktop program is tested apart from that selection, with the checks listed in Section 6.4: the same pixels under two different names give the same answer, a second `resize_and_crop` on a 256×256 crop does not change the array, an unreadable file keeps the previous window label, and the 60 test images replay through the program's own path without a single disagreement.

How do you ensure the robustness of the program in a real scenario?

Robustness is three different claims and they are supported to very different degrees, so they are answered separately rather than as one bullet list. The distinction matters because the first two are easy to demonstrate and the third is the one the project actually needs.

Pipeline consistency: the application and the evaluation are one object. This is measured. Evaluation and the desktop window both call `resize_and_crop` and both load `models/task3_pipeline.joblib`. The prediction was replayed over the whole test split through the program's own `predict_rgb`, returning the locked label on 60 of 60 images (Section 6.4). The classifier never sees the file name, so renaming a photograph cannot flip the prediction except by changing pixels; the same array under an `IN-` name and an `OUT-` name gives the same answer and the same probabilities. This shows the reported score belongs to the object the window loads. It is not evidence about accuracy on anything new.

Input handling: a bad file does not take the program down. An unreadable file keeps the previous window label and makes `--image` exit with status 1; a second `resize_and_crop` on an already-cropped 256×256 array returns the same array, so the crop cannot be applied twice by accident; the crop cache key includes the crop size, and a rerun stops when the split file or the training code has changed, so old crops cannot be silently mixed with new code. These are checks on behaviour, not on accuracy.

Predictive robustness on a place it has not seen: measured, and weak. This is the claim that matters and the one that is not satisfied. On the two campus places the model had not seen, inside recall is 0.545, and 15 of the 33 campus photographs are called outside; one of those two places, the CMA courtyard, is at 5/17, so the limit is not spread evenly over the campus. A real ``is this the main campus?'' decision that must not miss campus scenes is not supported by that recall. Three further axes are partly or wholly uncovered. Changes of lighting are not tested: the seven night frames in the test split are all inside frames from IN-05, so no night error rate can be estimated (Section 5.3). Lookalike buildings are covered by design rather than by a measurement: OUT-06 is the brick negative and the colour histogram that would have used that shortcut scored 0.537 on the location holdout and was not selected, and eleven of thirteen scene types occur in both classes. Portrait frames are a weak case on the evidence available: the crop keeps the short side and centre-crops, and the two 960×1080 course scenes in Section 6.4 are both classified correctly but with probabilities within 0.013 of one half, which is not a confident answer on that shape of frame.

What would raise the third claim is more unseen campus places under the same boundary rules, and a new held-out evaluation of this saved pipeline. The present test groups stay held out.

What problems do you find and how do you solve them?

Problem	What was done
GPS disappeared after the phones were copied to a computer.	Labels were anchored to signs visible in the frame (Section 2.3). GPS was never a feature.

Problem	What was done
Gate photographs are ambiguous and some show people.	11 IN-05 frames were excluded, with notes on the manifest.
Two Jordan frames show a readable plate; one Kowloon Park frame shows faces.	Those frames were excluded, and the decision was taken on the photograph.
Red brick can act as an inside shortcut.	HKU Main Building and part of Kowloon Park were collected as red outside scenes. The colour histogram then scored fifth of six on the location holdout (0.537), which is consistent with that design. Bag of words was last.
The validation pair is easy.	Validation is reported and is not the selection score.
The validation column and the holdout column pick different pairs: validation puts F3 second at 0.826, the holdout puts F2 second at 0.701.	The holdout score is the one that ranks features, so F3 never reached Stage B. Both columns are published.
k-NN's outside recall looks strong while its inside recall falls to 0.527 on F2.	Balanced accuracy is the score. The forest wins it without that collapse.
A crop cache keyed only by file name would reuse a 256×256 image after a size change.	The key includes the crop size, so a change of size invalidates the cache.
Group ids and place names were both derived from the collection folders rather than from the frames, and three grouping errors were found across four groups. That put the same scene on both sides of a split.	Place names were taken from the signs in the frames, and every frame of those four groups was re-examined and re-assigned by the building in the photograph; 31 frames moved and 2 were dropped. The split and the pipeline were rebuilt on the repaired data. Section 2.7 records the repair.
Test inside recall is 0.545.	Section 5 gives the breakdown. The pipeline was not retuned on the test set. The remedy is a new evaluation rather than a new fit: more inside places under the same boundary rules, one further held-out group, and the saved pipeline scored on it, as set out in Section 8.
A name such as `IN-99` could be read as the class.	`predict_rgb` receives the RGB array only. The same pixels under `IN-99_inside_campus.jpg` and `OUT-99_outside_campus.jpg` return the same label and the same probabilities (Section 6.4).

How do you use GenAI to assist you in the implementation?

I used AI for what it is good at: writing code quickly and running experiments quickly.

I started with information search. I read about how scene photographs are classified without deep learning, and collected a list of candidate representations: colour histogram, HOG, LBP, SIFT bag of words, spatial pyramid, together with the classifiers usually paired with them.

I then had the AI build the comparison code for that list. It wrote the feature extractors, the classifier wrappers, the five-fold location-holdout loop that refits the codebook and, where the classifier needs one, the scaler inside each fold, the dataset scripts that build the split table and check it, and the scripts that draw the figures. Each stage runs from one command.

What I decided myself: the protocol, written down before any result existed (five folds fixed in advance, one selection score, no tuning, the test set scored once on the repaired split), the place labels, which frames to keep or drop, and the photographs themselves whenever a generated note and the picture disagreed. The experiments then ran, and the choice followed the numbers rather than my expectation: the pair with the best location-holdout score won, and a feature that validation ranked second was dropped because it did not survive a change of place. After the selection I improved that model --- the refit on all training images, the saved pipeline, and the desktop program that loads it --- with the test set kept out of every decision.

The files the model drafted are `src/tools/build_dataset.py`, `src/tools/collapse_to_venues.py`, `src/tools/apply_naming.py`, the modules under `src/task3/`, and the figures under `docs/figures/`. They did not take the photographs, assign places, or decide include versus exclude; those decisions were made by looking at the frames.

Verification was reading the material back against the data, in two separate ways, and it is named here so that it can be repeated. The numbers were checked by recomputation: the sort key in `src/task3/cv.py` is the mean location-holdout balanced accuracy, the published Stage A and Stage B tables match the result tables of the locked run value for value, and the test confusion matrix recomputes from the per-image test predictions. The labels and the grouping were checked by eye: every frame of those four groups was re-read against the building in the photograph, and the sign and plate readings were taken from the image rather than from a generated note. Two of those checks appear above with the numbers attached: the five folds of the location holdout in Section 4.2, and the resolved tie between F4 random forest and F4 logistic regression in the same section. The 60-image replay in Section 6.4 is the third, and the only one run after the selection was locked.

How does GenAI understand and solve the tasks?

The generated code treats the work as a reproducible classical pipeline: parse a split file, crop, extract a feature, fit a scikit-learn classifier, write metrics. That part is sound, and it is the path that produced the Stage A and Stage B tables.

Its picture of the dataset was less reliable, and the errors show where the general recipe replaced the local rule:

- The campus-map annotation and the Figure 2 table were generated from a coarser place list of 12 inside groups, including ids IN-09, IN-10 and IN-12 that appear nowhere in `splits.csv`. The figure and the hard-negative section were rebuilt from the split file, so the figure, the table and the counts now come from one source.
- The CMA building was first recorded as 香港中華煤氣. The sign in the photograph reads 香港中華廠商聯合會樓 (CMA Building). The manifest entry was corrected from the sign.
- The characters of a license plate were taken from an annotation instead of from the image, and several of them were wrong. The value was corrected by reading the photograph, and the frame is excluded on privacy grounds regardless.
- Source manifest CSVs were left out of version control by a generated ignore rule, so only the derived `splits.csv` was tracked. The ignore rule was changed and the manifests were committed.

- When SIFT might be missing, the suggestion was to switch to ORB. The rule stated in Section 3.3 is to stop and revise the experiment, not to swap the descriptor quietly. The run kept SIFT because `cv2.SIFT_create` was present.

On the modeling choice, the generated tables put F4 and F2 first on the holdout and F4 and F3 first on validation. Reading the validation column as the answer would have sent the texture descriptor to Stage B instead of the shape descriptor. The selection rule had already defined the answer as location-holdout balanced accuracy. The tables were kept, including the validation column, and the holdout column was the one that ranked rows.

What are the limitations and areas for improvement in GenAI's solution?

The useful limit is that a generated pipeline will complete a standard recipe and will be wrong on facts that live only in the photographs or in a rule written for this campus.

- Text the assistant transcribed from an annotation rather than from the image was wrong, and the place names drifted when the list underneath them changed. The improvement is to treat any fact about image content or place identity that the assistant writes down as a claim to be checked against the file, and to regenerate the map from the split file rather than from the map, which is what the rewrite did.
- A substitution of ORB for SIFT would have made the experiment irreproducible: the descriptor would have changed under the same name. The improvement is a hard stop, which is what the code does when SIFT is absent.
- A table of numbers invites a ranking, and the validation column ranked a different second feature from the holdout column. The safeguard adopted here is to freeze the selection column before the run and to publish the other column so the disagreement stays visible.
- The test set is scored once on the repaired split. Refitting on IN-05 or IN-07 to raise the 0.733 would break the holdout, and no such fit was made. That rule was fixed before the run rather than derived from the result. The qualification is that the per-place rates for these four places had been computed and read before the split was repaired, so this is a corrected evaluation rather than a blind one.
- A location group is a claim about the photographs, and a claim made from a folder name was wrong for four groups. The improvement is to check the claim against the photograph: every frame of those four groups was re-read, and what the sheet showed is in Section 2.7.

The model-side limit in Section 5 is separate. It was measured on the locked pipeline on unseen places, and the measurements do not identify which property of the representation is responsible. It is not repaired by another round of generation against the test score.

---

8. Reflection

What I expected from a protocol fixed in advance was a ranking that would hold up, and a score that would carry to new places. The first expectation was only partly right, and the second was wrong.

The protocol did what it was built to do at the level of the rules: the colour shortcut was rejected, the codebook was rejected, and no classifier was chosen by validation accuracy. What the rules did not control was how much the score moves between folds. With one seed and effectively four independent inside training places, the fold-to-fold spread is wider than the gaps between the top configurations, so the order of the top rows is not something this much data can settle.

What the result taught is that the score carries less than the design assumed. The selected pair, HOG with a random forest at 0.722 location-holdout balanced accuracy, becomes accuracy 0.733 and inside recall 0.545 on the held-out test places, with OUT-06 at 0.950, OUT-07 at 1.000, IN-05 at 0.812, and IN-07 at 0.294. The outside places transferred almost perfectly and one campus place did not. The holdout had suggested a weaker version of the same asymmetry, with inside recall 0.622 against outside recall 0.821, but nothing in it suggested a drop to 0.294 on the hardest of the four places.

Those test numbers are a usable measurement and a poor campus detector. The representation has no separate channel for ``this is PolyU'' once colour is set aside and the building has not been seen. What transfers is local gradient, and the failure is consistent with a façade that resembles nothing in the training places, which is what the CMA courtyard is; that reading is a hypothesis, not a measurement. Whether the limit is the material or the layout is not settled here, and the test set was not refit to find out. A second lesson came from the repair in Section 2.7: a location group is a claim about photographs, and a claim made from a folder name was wrong for four of the sixteen, so the groups had to be re-read and the whole pipeline rebuilt before any of the numbers above could be called a measurement.

What would actually change the evidence is more inside places, collected under the same boundary rules, and one new held-out evaluation of this saved pipeline. The route is concrete: add inside places so that a fifth inside group can be held out, leave the current IN-05 and IN-07 groups in the test set and never train on them, and score the new group with the pipeline saved in `models/task3_pipeline.joblib` rather than with a refit.

Two further measurements are worth making, and they cost the current result very different amounts.

The cheap one is a per-cell breakdown that also splits by scene class. It reads the 60 predictions that already exist, adds no fit, and does not touch the locked model; the aggregate table cannot show which part of a frame the model actually saw.

The expensive one is a feature that keeps a cue the locked one throws away. HOG is computed on the grayscale crop, so colour is not available to it, and a variant that adds a colour histogram would separate ``the crop was mostly brick and paving'' from ``the negative was never learnable'' (Section 5.4 shows the misses clustering on crops where the identifying mark is missing or replaced by something adjacent). **That variant is a new pipeline, not a reading of the current one.** It needs its own fit, its own location-holdout selection and its own held-out evaluation, and it would replace the saved model rather than extend it. `models/task3_pipeline.joblib` stays the baseline the candidate is measured against, and the 0.733 in Section 4.4 is the baseline's number, not the candidate's.

---

9. References

1. N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
2. T. Ojala, M. Pietikäinen, and T. Mäenpää. Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7):971--987, 2002.
3. L. Breiman. Random forests. *Machine Learning*, 45(1):5--32, 2001.

4. D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91--110, 2004.
5. G. Csurka, C. R. Dance, L. Fan, J. Willamowski, and C. Bray. Visual categorization with bags of keypoints. In *ECCV Workshop on Statistical Learning in Computer Vision*, 2004.
6. S. Lazebnik, C. Schmid, and J. Ponce. Beyond bags of features: Spatial pyramid matching for recognizing natural scene categories. In *CVPR*, 2006.
7. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825--2830, 2011.
8. S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, T. Yu, and the scikit-image contributors. scikit-image: Image processing in Python. *PeerJ*, 2:e453, 2014.
9. G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.
10. The Hong Kong Polytechnic University. Campus map. `https://www.polyu.edu.hk/campus-map/`. Geographic reference for the inside class.

---

10. Figure index

All ten image files are in `docs/figures/` in the code archive.

Figure	File	Role
1	`pipeline.png`	The pipeline and the order in which the model was chosen.
2	`figure1_inside_locations_annotated.png`	The seven inside places on the PolyU campus map, one numbered pin per place. The numbers, group ids, splits and counts are in the table in Section 2.1.
3	`location_overview.png`	All sixteen collection locations: nine outside groups on a map of Hong Kong and the seven inside groups on a campus inset. Colour is the class, shape is the split.
4	`task4_val_confusion_matrix.png`	Validation confusion matrix, locked pipeline.
5	`task4_test_confusion_matrix.png`	Test confusion matrix, one evaluation.
6	`task4_test_per_group.png`	Test accuracy by place.

Figure	File	Role
7	`task4_example_grid.png`	Thirteen test photographs, one per selected cell, labelled with the place and the outcome. The file list is in `task4_example_manifest.csv`.
8	`task5_application_inside.png`	The application running with a campus photograph loaded. A screenshot of the program itself.
9	`task5_application_outside.png`	The application running with an outside photograph loaded. A screenshot of the program itself.
10	`task5_application.png`	The prediction path applied to one photograph: the file, the 256×256 crop the model receives, and the label and probabilities it returned.

---

11. Appendix: terms and abbreviations

The report uses short names for the feature sets, the classifiers and a number of standard ideas in image analysis. This section says what each one means here, so that a reader who has not worked with these particular methods can follow Sections 3 to 5.

11.1 The feature sets

A feature set turns one photograph into a fixed-length vector of numbers. `F1` to `F6` are the six that were compared. `F2` is the one the locked model uses; the other five are comparison features. `F4` was the runner-up in Stage B, where its best classifier reached 0.712 against 0.722 for `F2`; under the linear SVM of Stage A it ranked first, at 0.708 against 0.701.

Id	Short name	What it measures	Values
F1	HSV histogram	How much of each colour and brightness band the frame contains	10240
F2	HOG	How the brightness gradient points are oriented, in 9 direction bins, over 16×16 cells pooled in 2×2 blocks. **The locked feature set**	8100
F3	LBP	A 10-bin histogram of the local binary pattern, each pixel compared with its 8 neighbours, on the whole frame and on a 2×2 grid	50

Id	Short name	What it measures	Values
F4	HOG + LBP	F2 and F3, each standardized on the training images and then concatenated. Runner-up on the location holdout (0.708 / 0.712), not used by the locked model	8150
F5	Bag of words	SIFT keypoints pooled into 256 visual words by k-means and counted	256
F6	Bag of words + spatial pyramid	F5 on the whole frame, plus four histograms on a 2×2 grid	1280

11.2 The classifiers

Id	Name	What it does here
C1	Linear SVM	A linear maximum-margin classifier. The fixed classifier of the feature comparison, so that stage one varies only the feature
C2	Logistic regression	A linear model whose output can be read as a probability
C3	k-NN	Classifies a photograph by the labels of its 5 nearest neighbours in feature space
C4	Random forest	300 decision trees voting on the answer. **The locked classifier**
C5	Histogram gradient boosting	Boosts small trees over histogram-binned features
C6	Multinomial naive Bayes	A count-based classifier. Valid only for non-negative histograms, so it applies to F1, F3, F5 and F6 and never reaches the two features that advanced

11.3 Features and preprocessing

Term	Meaning here
Augmentation	Mirroring the training photographs to enlarge the training set. Validation and test images are never mirrored
Centre crop	Cutting the middle square out of a scaled frame. A landscape frame loses equal strips on the left and the right; a portrait frame loses them at the top and the bottom
Gray	The single-channel version of the crop, used by HOG, LBP and SIFT
HSV	Hue, saturation and value. The colour space the colour histogram is computed in
L1 normalisation	Making a histogram sum to 1, so that it describes proportions rather than raw counts
L2-Hys	The HOG normalisation that also clips the largest gradients, so that one strong edge cannot dominate a cell
P and R	The local binary pattern's neighbourhood size and radius, here P = 8 and R = 1

Term	Meaning here
Pipeline	The fitted scikit-learn object holding the feature extractor and the classifier together, so that training and the application run the same code
SIFT	Scale-Invariant Feature Transform, the local keypoint detector used by F5 and F6
Spatial pyramid	Splitting the frame into regions and histogramming each separately, so that where something appears matters and not only how much of it there is
Standardisation	Subtracting the mean and dividing by the standard deviation, using the training images only
uint8	Integers from 0 to 255, the pixel format the crop is returned in

11.4 Evaluation

Term	Meaning here
Balanced accuracy	The mean of the two class recalls. A constant predictor scores 0.500 on it, which is why it is the selection score
Confusion matrix	The 2×2 table of true class against predicted class
Fold	One repetition of the location holdout. There are five. Each holds out one inside place and at least one outside place; fold 4 holds out IN-01 together with OUT-05 and OUT-08, because those two outside places can be seen from each other
Hard negative	An outside-class photograph that looks like the inside class. Here the Hong Kong University buildings, collected so that the model cannot read brick as a campus cue
Inside recall	The share of campus photographs correctly called inside
Location holdout, also called venue holdout	A cross-validation in which every fold holds out whole places, so the model is always measured on places it has not seen
Precision	Of the photographs called inside, the share that really are inside
Recall	Of the photographs that really are inside, the share that are called inside
F1 score	The harmonic mean of precision and recall. Not to be confused with the feature set F1 in Section 11.1
Support	The number of test photographs in a class
Majority-class baseline	A predictor that always answers the more common class of the test set

11.5 The data and the split

Term	Meaning here
group_id	The identifier of a physical place. It is the unit that is split, so no place appears in two splits

Term	Meaning here
Inside (1)	A photograph taken on the PolyU main campus, with the boundary taken from the official campus map
Outside (0)	A photograph taken anywhere else, including the campuses of other institutions
Seed	The fixed random seed, 42, used for the tree ensembles and the codebook. The five folds are not drawn at random: they are written out in `src/task3/config.py`
Split	One of train, validation or test. The validation split is reported but never used to choose anything

11.6 The sizes the split rule read

The repair of Section 2.7 moved 31 frames between groups and excluded 2. It changed the split of three groups and the stored size of five, because the assignment rule reads the size.

Group	Split before the repair	Split after	Frames before	Frames after
IN-03	test	train	25	44
IN-05	validation	test	38	16
IN-06	train	validation	40	40
OUT-04	train	train	30	29
OUT-08	train	train	30	29
OUT-09	validation	validation	30	29

The other ten groups kept both their split and their stored count. The moves close group by group:

- IN-03 gained 19: 13 frames from IN-05 and 6 from IN-01, so 25 becomes 44.
- IN-05 lost 22: 13 to IN-03 and 9 to IN-01, so 38 becomes 16.
- IN-01 gained 9 and lost 8, six of them to IN-03 and two excluded, so it stays at the per-group cap of 50.

The counts do not map one-to-one onto the move list. Only two frames left the data, and both came out of IN-01, which is capped at 50: its usable pool rises from 71 to 72 and it still keeps 50, so the removals cost no kept frame. After the moves IN-03 keeps 44 and IN-05 keeps 16 where they kept 25 and 38, which is three fewer on the inside side, and that lower inside total is what the balance step re-reads: the target becomes 267, whose even share over nine outside groups is 29 rather than the 30 of a target of 270, so the three outside groups that had been kept at exactly 30 give up one frame each. Three and three make 534.

None of this was edited by hand on the split table. `src/tools/recut_locations.py` holds the table of moves and exclusions, and `build_dataset.py` re-derived the split from the repaired manifests.